CS 322: COMPUTER VISION

Vision Transformers



COMP 322: Computer Vision and Image Processing

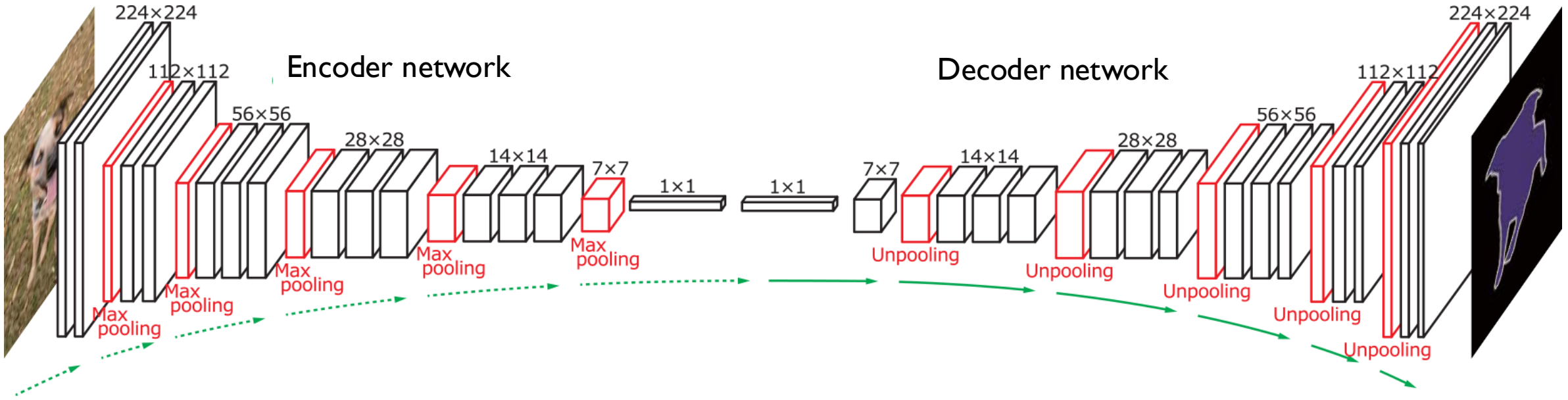
Hong Xu · Rhodes College

Adapted from Brown University CSCI 1430 Computer Vision (Spring 2026).

Original file: 2026_Spring_L19_Transformers.pptx. Also credits Prof. Simon Prince / Understanding Deep Learning where noted in-deck.

Image Credit: Pixar Animation Studios

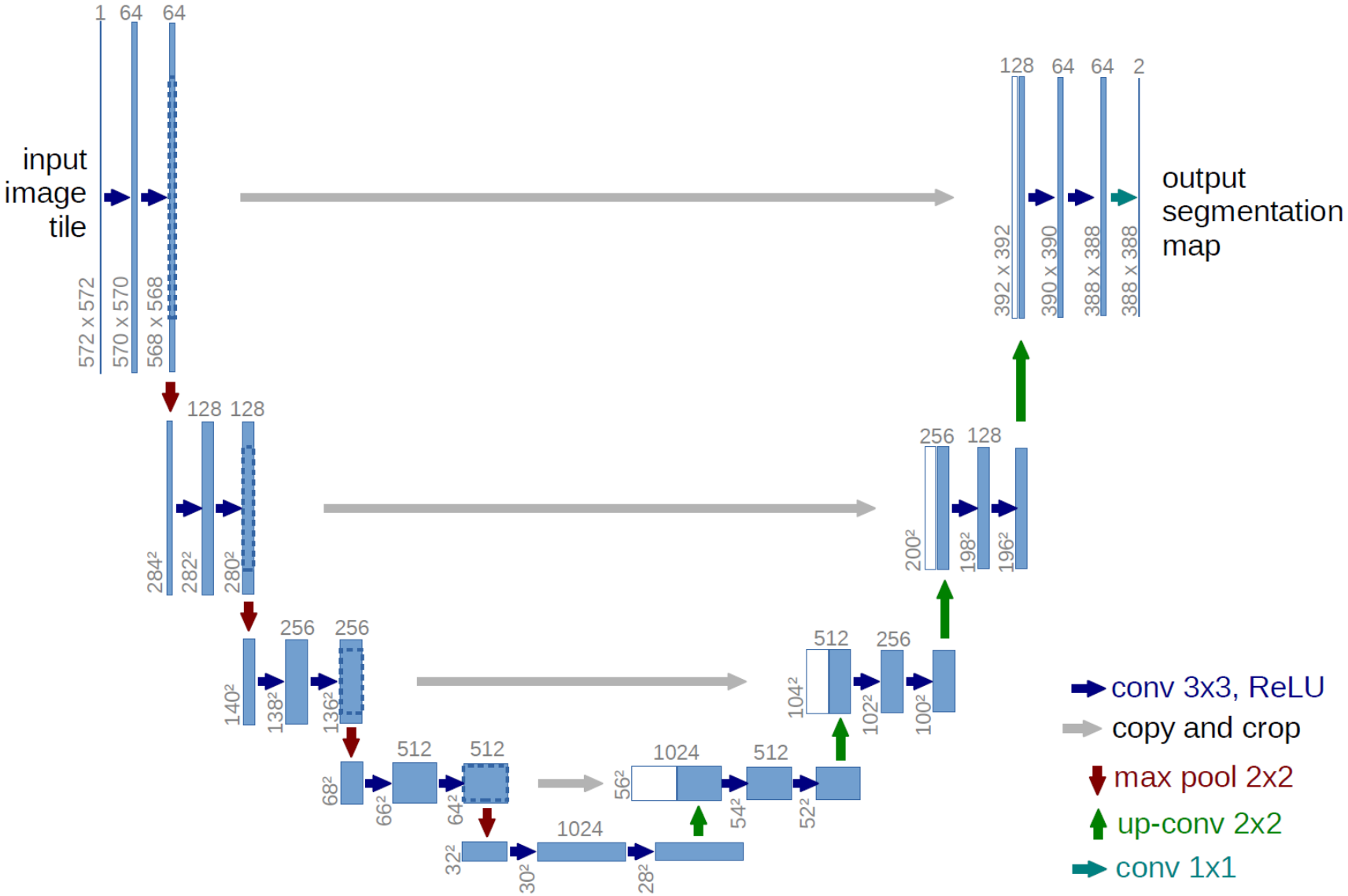
Decoders, Encoders, Autoencoders



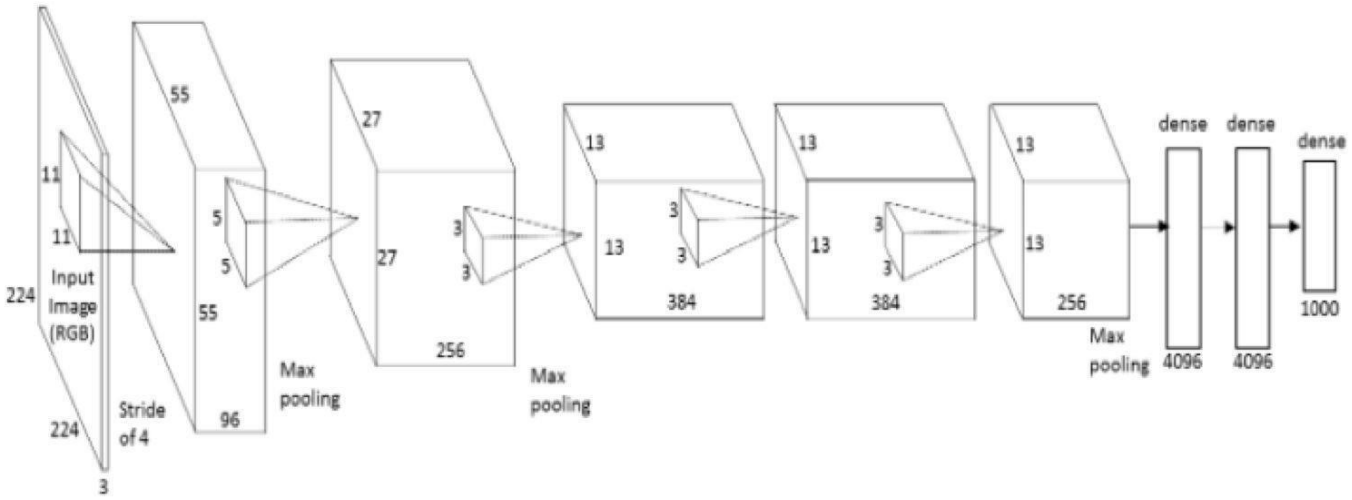
Zeiler et al., Deconvolutional Networks, CVPR 2010

Noh et al., Learning Deconvolution Network for Semantic Segmentation, ICCV 2015

UNet [Ronneberger et al., 2015]



What input to a neuron maximizes a class score?

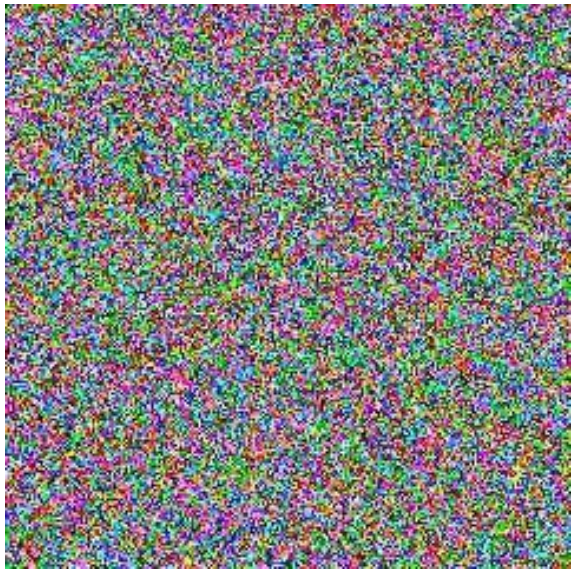


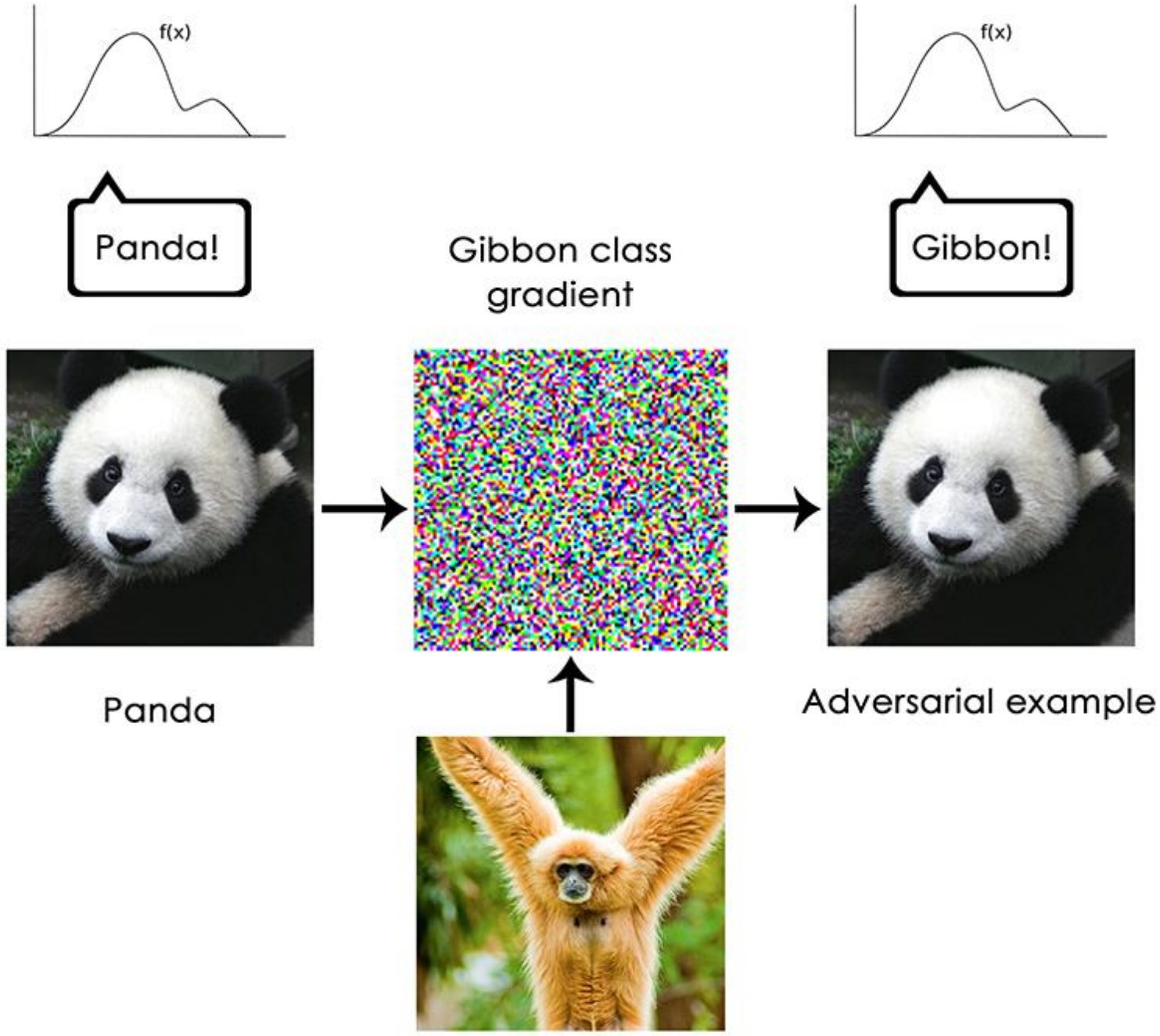
To visualize the function of a specific unit in a neural network, we synthesize an input to that unit which causes high activation.

Neuron of choice *i*
An image of random noise *x*.

Repeat:

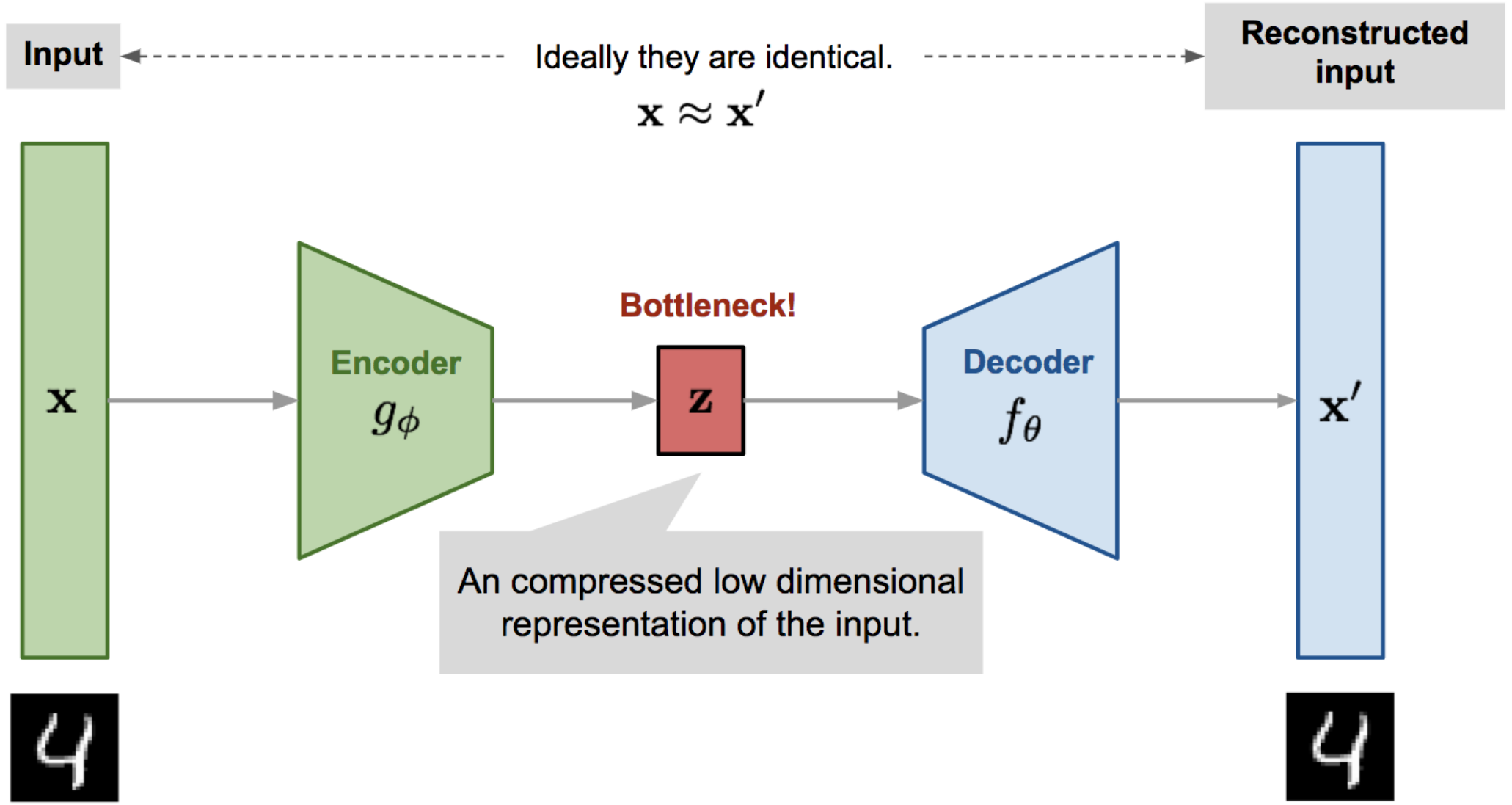
1. Forward propagate: compute activation $a_i(x)$
2. Back propagate: compute gradient at neuron $\partial a_i(x) / \partial x$
3. Add small amount of gradient back to noisy image.





Francois Chollet - https://blog.keras.io/the-limitations-of-deep-learning.html

Autoencoders



Noise is learned *last! (1x1 conv, bilinear, ReLU)*

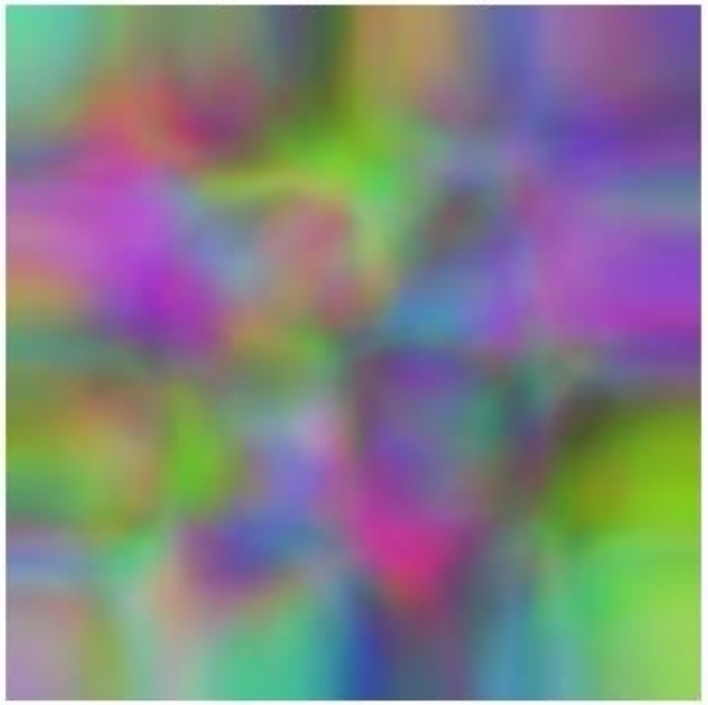
1 x 1 convolutions have fewer free parameters, more constrained.
Fits less noise than 3x3; achieves higher PSNR to clean target.

Step 1/15000 | PSNR: 9.4 dB | lr: 0.00000

Noisy input



Original_1x1 (step 1)



Clean target



Note: Uses cosine annealing learning rate decay.

Many slides from:

Prof. Simon Prince @ UBath

Understanding Deep Learning (course book!)



Some diagrams are subject to a Creative Commons CC-BY-NC-ND license. (C) MIT Press.

Motivation originally in text

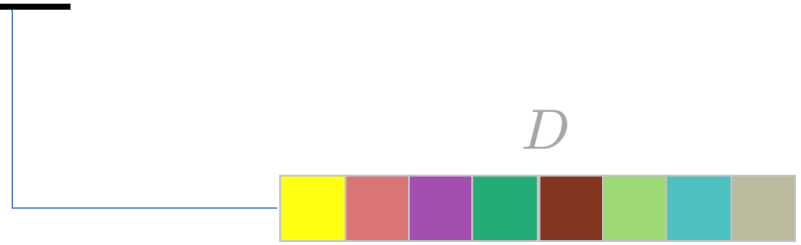
Design neural network to encode and process sequences:

The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.

Motivation

Design neural network to encode and process sequences:

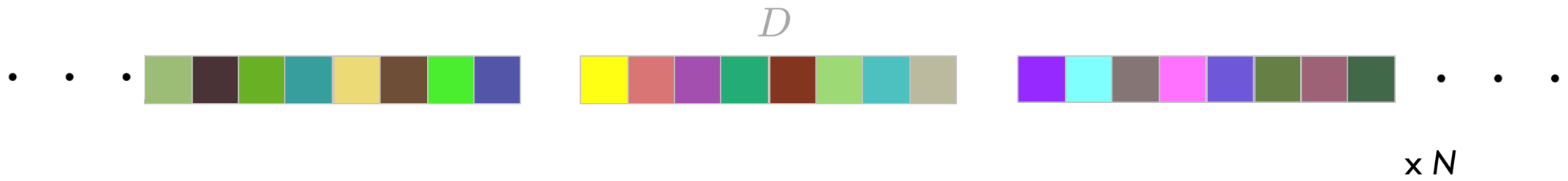
The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.



Motivation

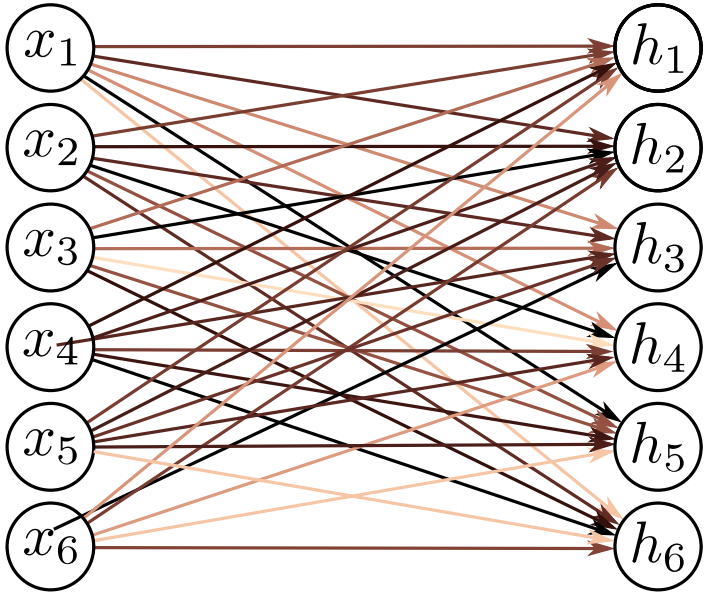
Design neural network to encode and process sequences:

The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.



Standard fully-connected layer

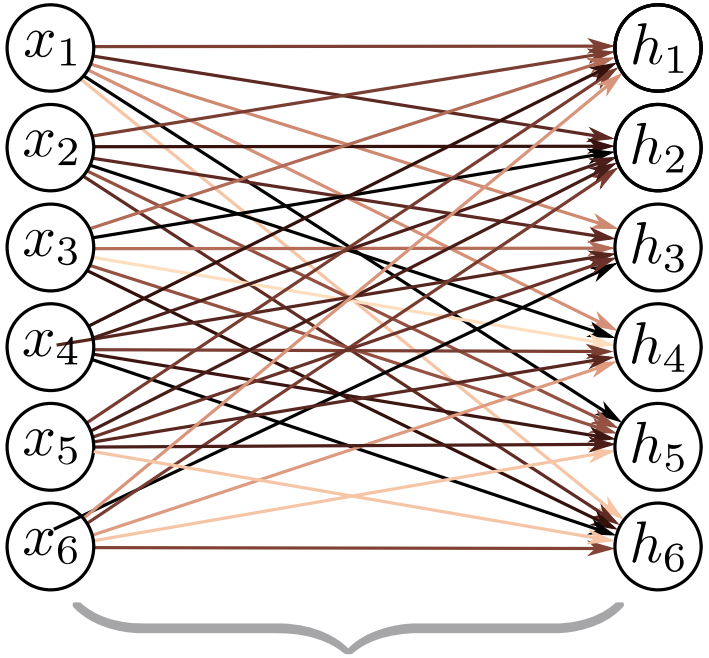
$$\mathbf{h} = \mathbf{a}[\boldsymbol{\beta} + \boldsymbol{\Omega}\mathbf{x}]$$



$\mathbf{x}$ inputs
$\boldsymbol{\Omega}$ weights
$\boldsymbol{\beta}$ biases
$\mathbf{a}$ activation
$\mathbf{h}$ hidden layer

Standard fully-connected layer

$$\mathbf{h} = \mathbf{a}[\boldsymbol{\beta} + \boldsymbol{\Omega}\mathbf{x}]$$



$\mathbf{x}$ inputs
$\boldsymbol{\Omega}$ weights
$\boldsymbol{\beta}$ biases
$\mathbf{a}$ activation
$\mathbf{h}$ hidden layer

Standard fully-connected layer

$$\mathbf{h} = \mathbf{a}[\boldsymbol{\beta} + \boldsymbol{\Omega}\mathbf{x}]$$

Problem:

- A very large number of parameters
- Can't cope with sequences of different lengths

Conclusion:

- We need a model where parameters don't increase with input length

Motivation

Design neural network to encode and process sequences:

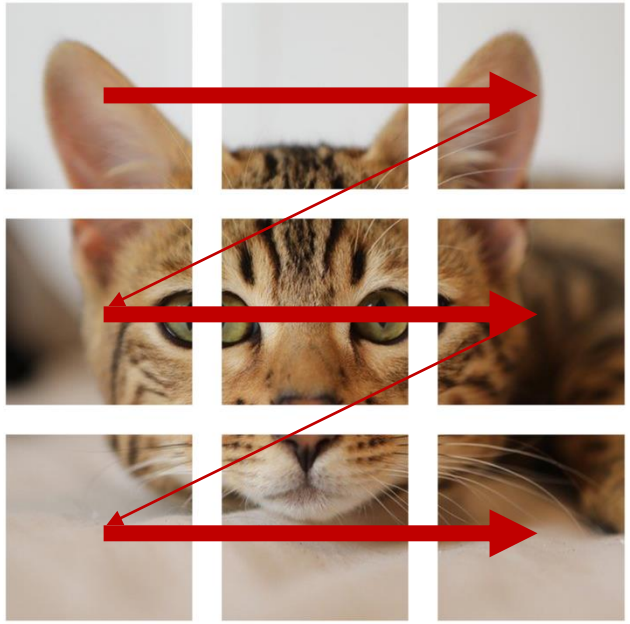
The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.

The word their must "attend to" the word restaurant.

Motivation

Design neural network to encode and process sequences:

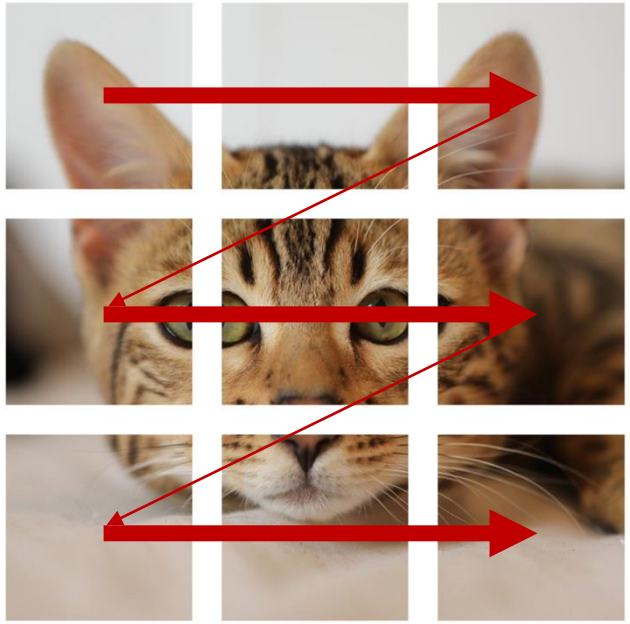




Motivation

Design neural network to encode and process sequences:





The ear must "attend to" the other ear, eyes, and nose to know it is a cat.

- There must be connections between the elements.
- The strength of these connections will depend on the elements themselves.

Dot-product self attention

- Takes N inputs of size Dx1 and returns N inputs of size Dx1
- Computes N values (no ReLU)

$$\mathbf{v}_n = \boldsymbol{\beta}_v + \boldsymbol{\Omega}_v \mathbf{x}_n$$

Dot-product self attention

- Takes N inputs of size Dx1 and returns N inputs of size Dx1
- Computes N values (no ReLU)

$$\mathbf{v}_n = \boldsymbol{\beta}_v + \boldsymbol{\Omega}_v \mathbf{x}_n$$

- N outputs are weighted sums of these values

$$\mathbf{sa}[\mathbf{x}_n] = \sum_{m=1}^{N} a[\mathbf{x}_n, \mathbf{x}_m]\mathbf{v}_m$$

Dot-product self attention

- Takes N inputs of size Dx1 and returns N inputs of size Dx1
- Computes N values (no ReLU)

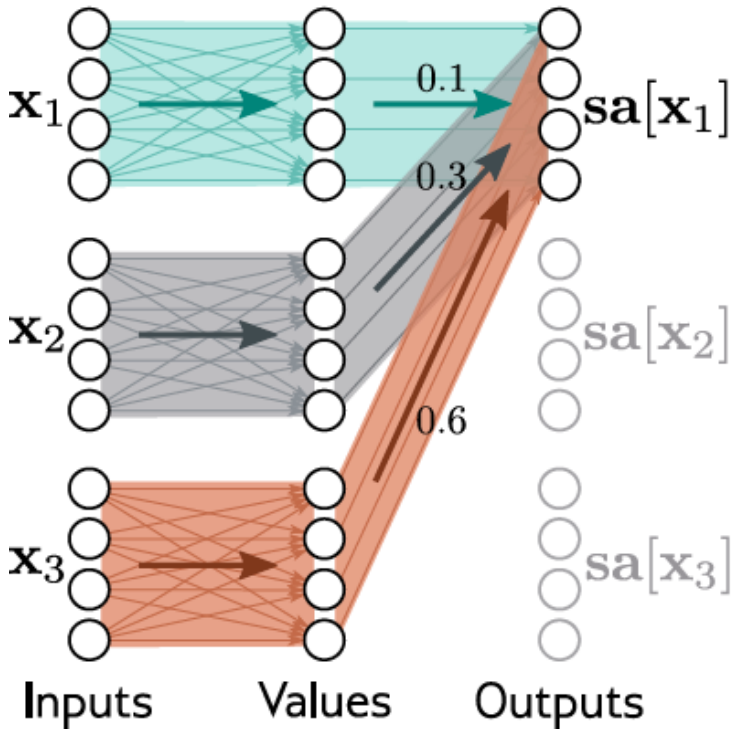
$$\mathbf{v}_n = \boldsymbol{\beta}_v + \boldsymbol{\Omega}_v \mathbf{x}_n$$

- N outputs are weighted sums of these values

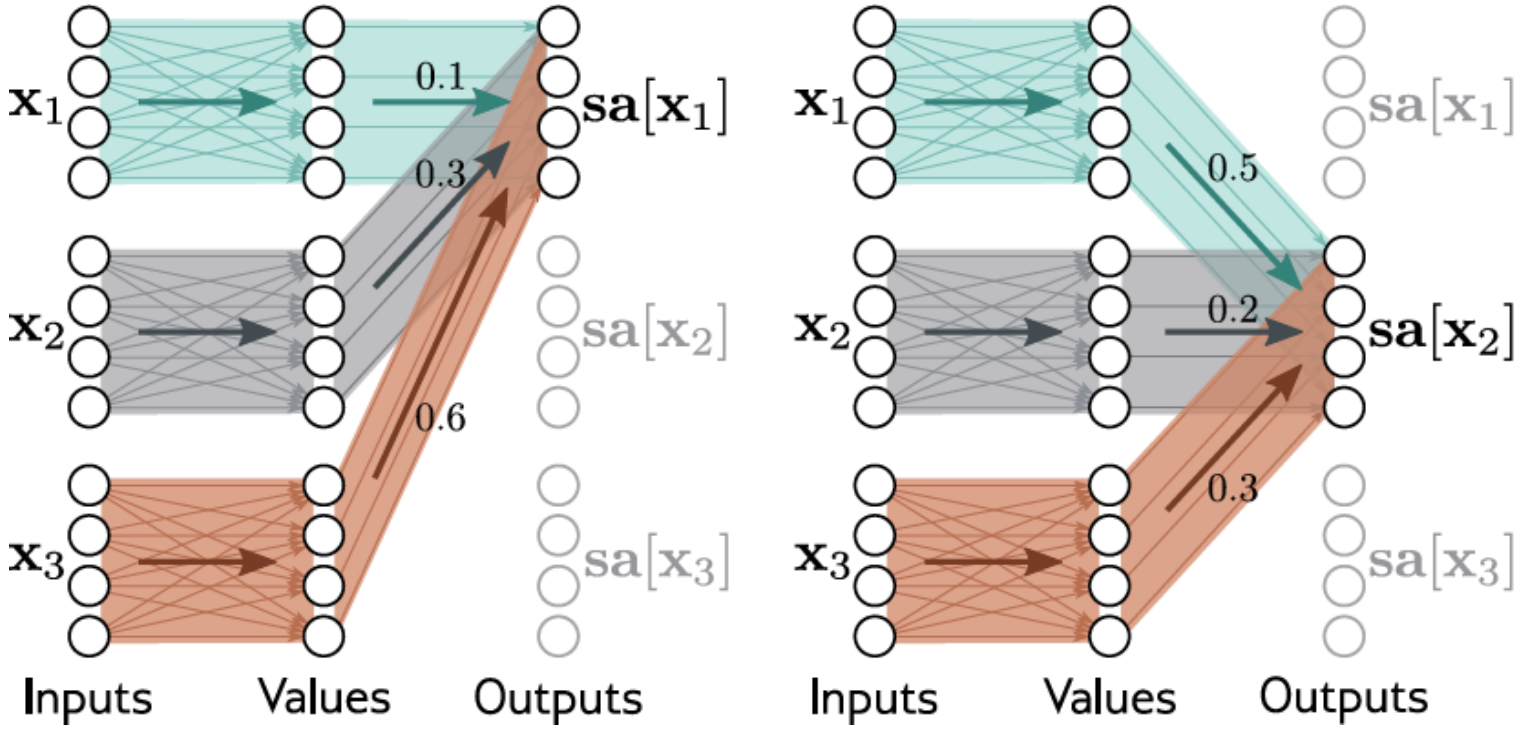
$$\mathbf{sa}[\mathbf{x}_n] = \sum_{m=1}^{N} a[\mathbf{x}_n, \mathbf{x}_m]\mathbf{v}_m$$

- Weights depend on the inputs themselves

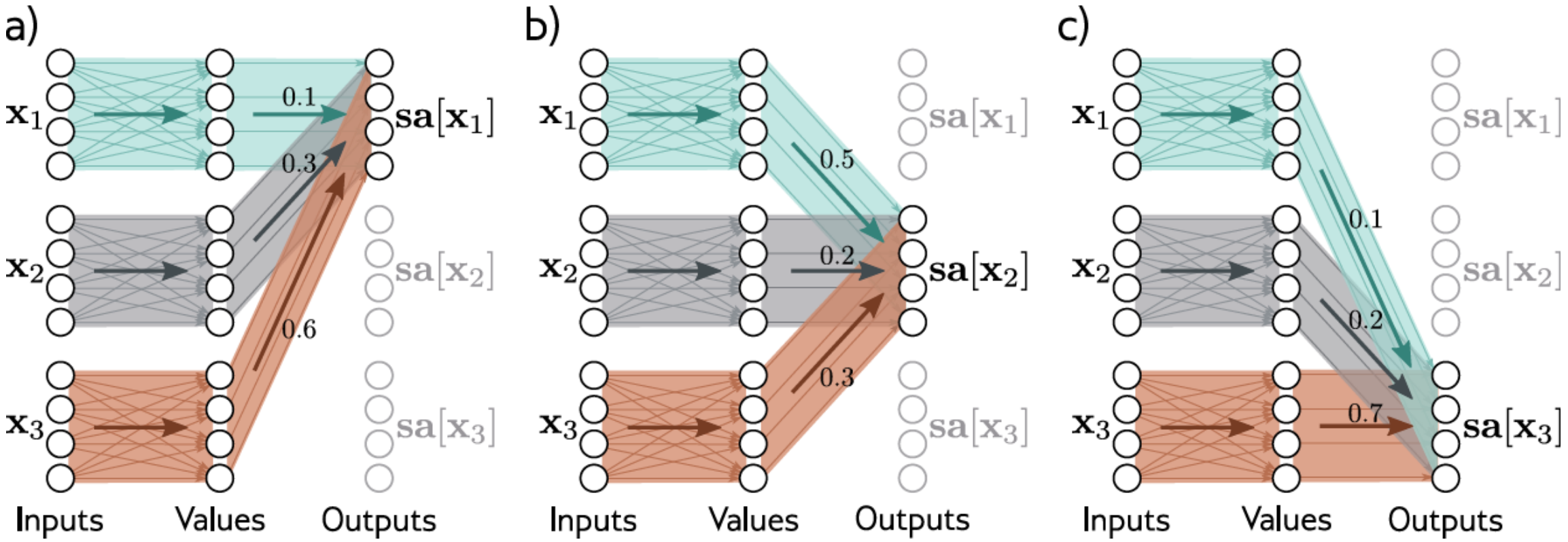
Attention as routing



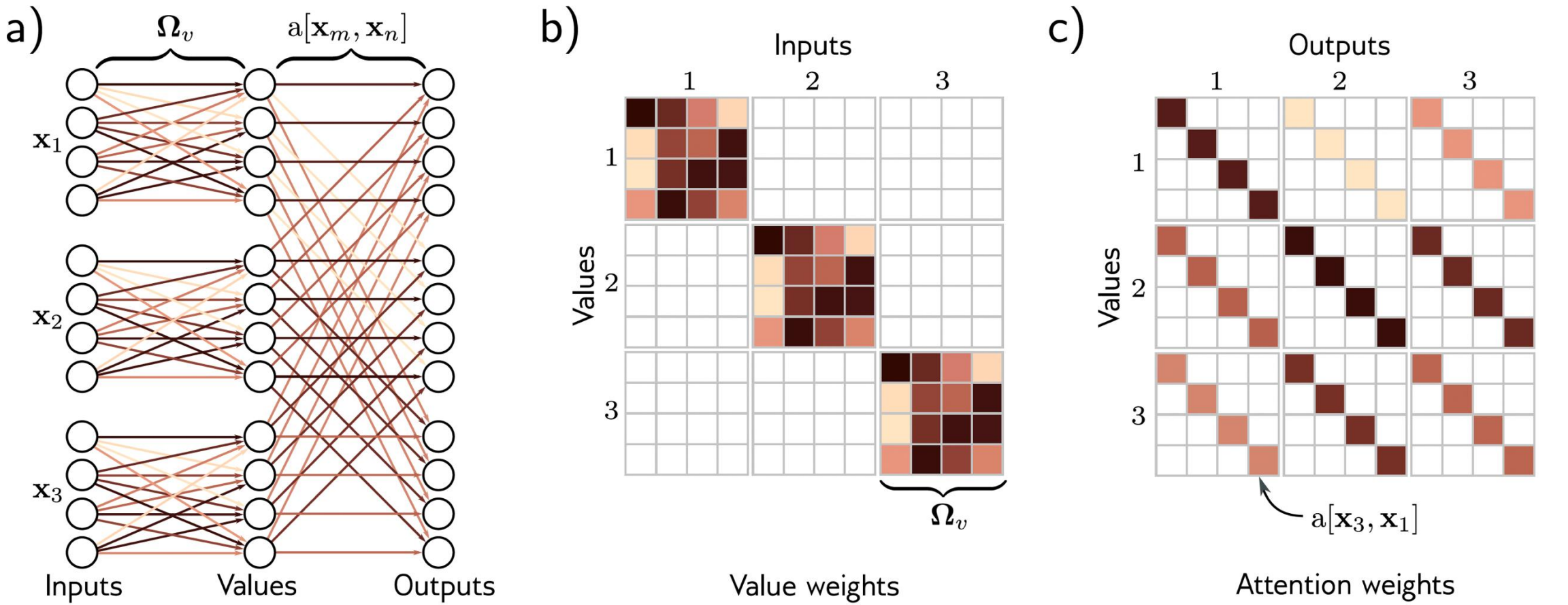
Attention as routing



Attention as routing



Attention as matrix view



Same color = same weight

Attention weights

- Compute N "queries" and N "keys" from input

$$\mathbf{q}_n = \boldsymbol{\beta}_q + \boldsymbol{\Omega}_q \mathbf{x}_n$$
$$\mathbf{k}_n = \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathbf{x}_n,$$

- Calculate similarity and pass through softmax:

$$a[\mathbf{x}_n, \mathbf{x}_m] = \text{softmax}_m\left[\text{sim}[\mathbf{k}_m \mathbf{q}_n]\right]$$
$$= \frac{\exp\left[\text{sim}[\mathbf{k}_m \mathbf{q}_n]\right]}{\sum_{m'=1}^{N} \exp\left[\text{sim}[\mathbf{k}'_m \mathbf{q}_n]\right]},$$

Attention weights

- Compute N "queries" and N "keys" from input

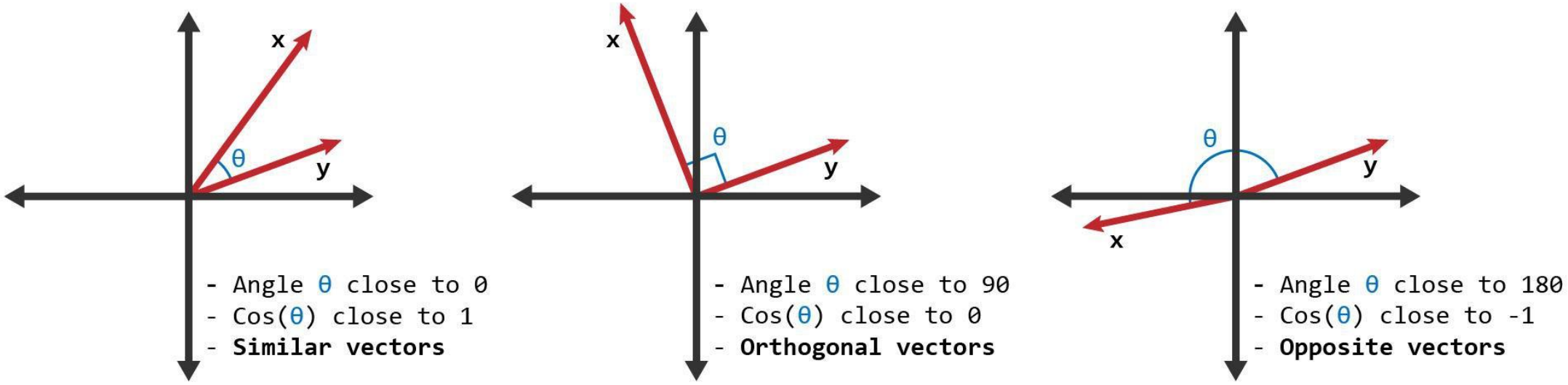
$$\mathbf{q}_n = \boldsymbol{\beta}_q + \boldsymbol{\Omega}_q \mathbf{x}_n$$
$$\mathbf{k}_n = \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathbf{x}_n,$$

- Take dot products and pass through softmax:

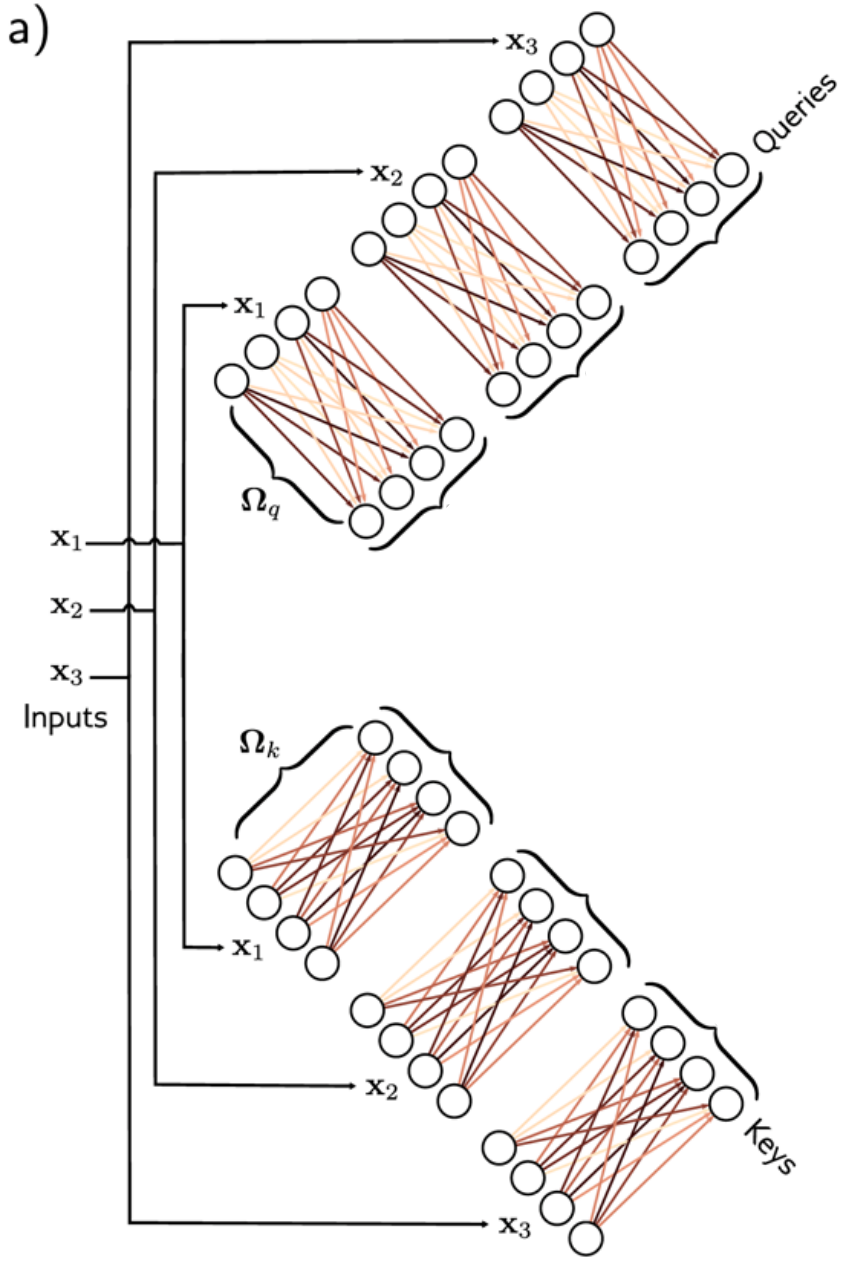
$$a[\mathbf{x}_n, \mathbf{x}_m] = \text{softmax}_m\left[\mathbf{k}_m^T \mathbf{q}_n\right]$$
$$= \frac{\exp\left[\mathbf{k}_m^T \mathbf{q}_n\right]}{\sum_{m'=1}^{N} \exp\left[\mathbf{k}_{m'}^T \mathbf{q}_n\right]}$$

Recap: Dot product = measure of similarity

$$\mathbf{x}^T\mathbf{y} = |\mathbf{x}| \cdot |\mathbf{y}| \cdot \theta$$



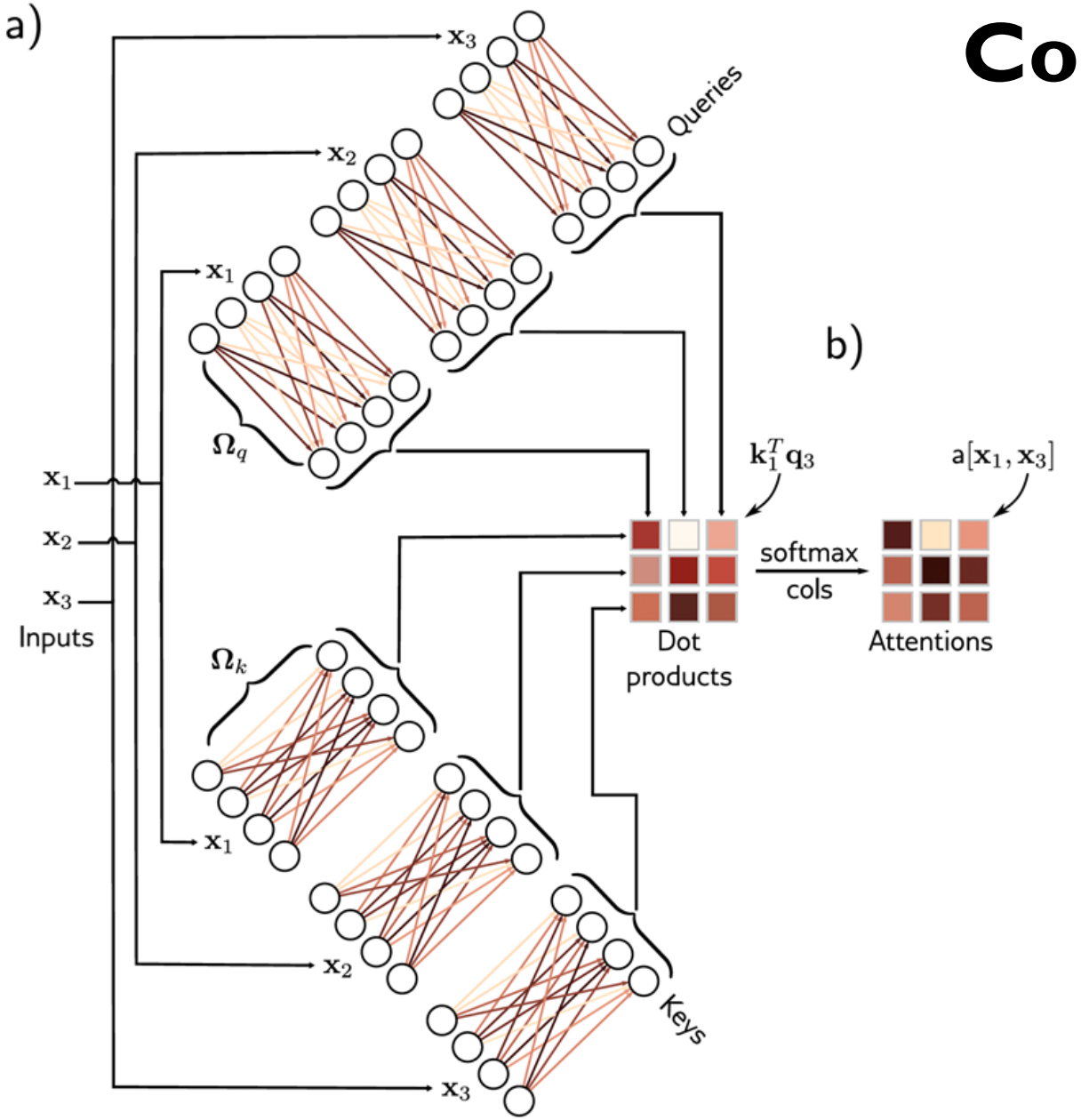
Computing attention weights



Same color = same weight

a) Queries and keys computed for each input via their own weights $\boldsymbol{\Omega}$ and bias $\boldsymbol{\beta}$ (hidden).

Computing attention weights

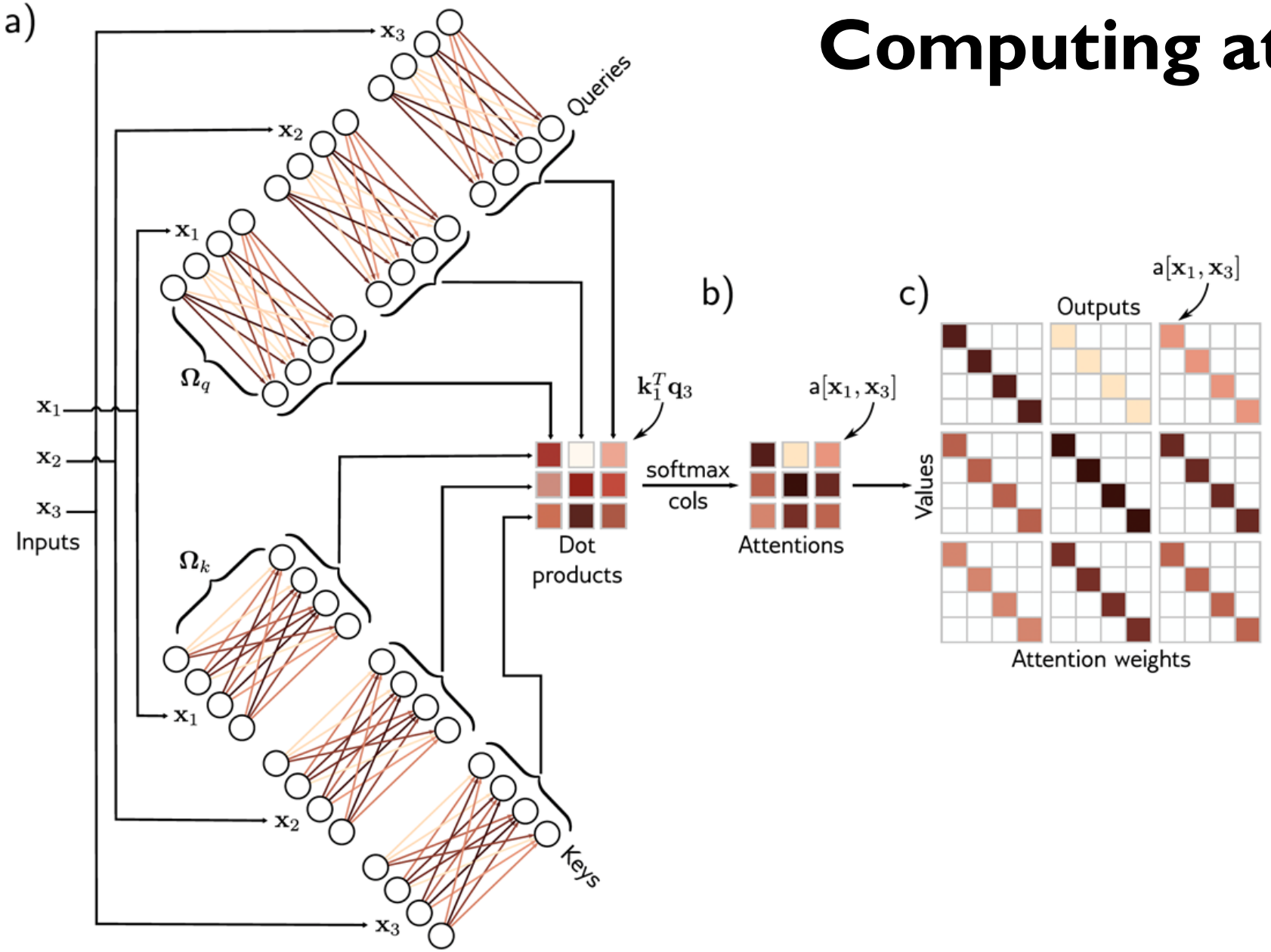


a) Queries and keys computed for each input via their own weights $\boldsymbol{\Omega}$ and bias $\boldsymbol{\beta}$ (hidden).

b) Dot product between all queries and keys.
Softmax over columns
(non-negative; sum to one)

Same color = same weight

Computing attention weights



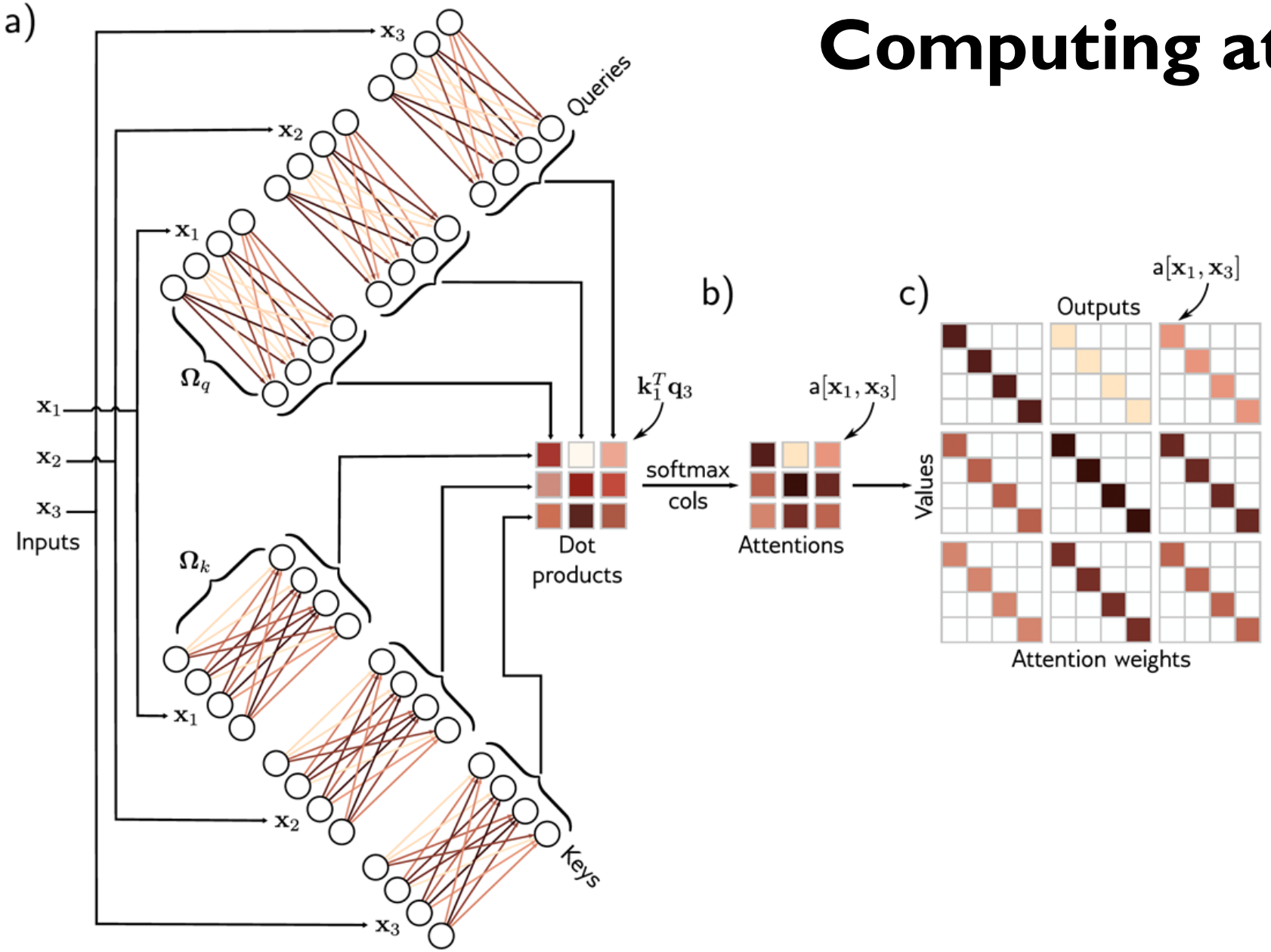
a) Queries and keys computed for each input via their own weights $\boldsymbol{\Omega}$ and bias $\boldsymbol{\beta}$ (hidden).

b) Dot product between all queries and keys. Softmax over columns (non-negative; sum to one)

c) Routing visualization of value vectors by attention.

Same color = same weight

Computing attention weights



a) Queries and keys computed for each input via their own weights $\boldsymbol{\Omega}$ and bias $\boldsymbol{\beta}$ (hidden).

b) Dot product between all queries and keys.
Softmax over columns
(non-negative; sum to one)

c) Routing visualization of value vectors by attention.

Same color = same weight

Properties of self-attention

- Permutation equivariant – operates on *sets* of vectors (without PE – *later*)
- Complexity is $O(n^2 d)$ where n is input size, d is input dimension
- Every position attends to every other position in one step
  No iteration or recurrence

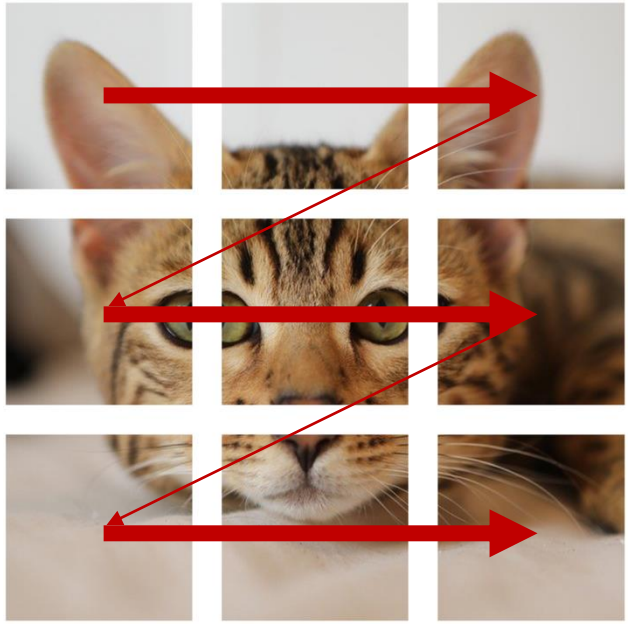
Contrast with convolution:

- Translation equivariant
- Complexity is $O(knd^2)$ where k is kernel size
- Must use receptive field across $O(\log_k(n))$ layers to 'see' all positions

Motivation – did we achieve it?

Design neural network to encode and process sequences:





- We have a model where parameters don't increase with input length

$$\phi = \{\boldsymbol{\beta}_v, \boldsymbol{\Omega}_v, \boldsymbol{\beta}_q, \boldsymbol{\Omega}_q, \boldsymbol{\beta}_k, \boldsymbol{\Omega}_k\}$$

- We have connections between the elements.
- The strength of these connections will depend on the elements themselves.

Self Attention Details

Scaled dot product self attention:

Prevents large magnitudes in dot products dominating softmax (causing slow training).

Regular self attention

$$\text{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}[\mathbf{K}^T\mathbf{Q}]$$

Scaled self attention

$$\text{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}\left[\frac{\mathbf{K}^T\mathbf{Q}}{\sqrt{D_q}}\right]$$

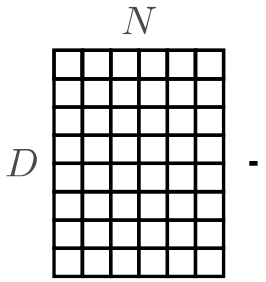
Dimension (queries)

Masked self attention for training

- Set all attention values 'ahead of' query to –inf.
- Stops method 'cheating' and looking ahead in sequence.

Matrix form

- Store N input vectors in matrix X



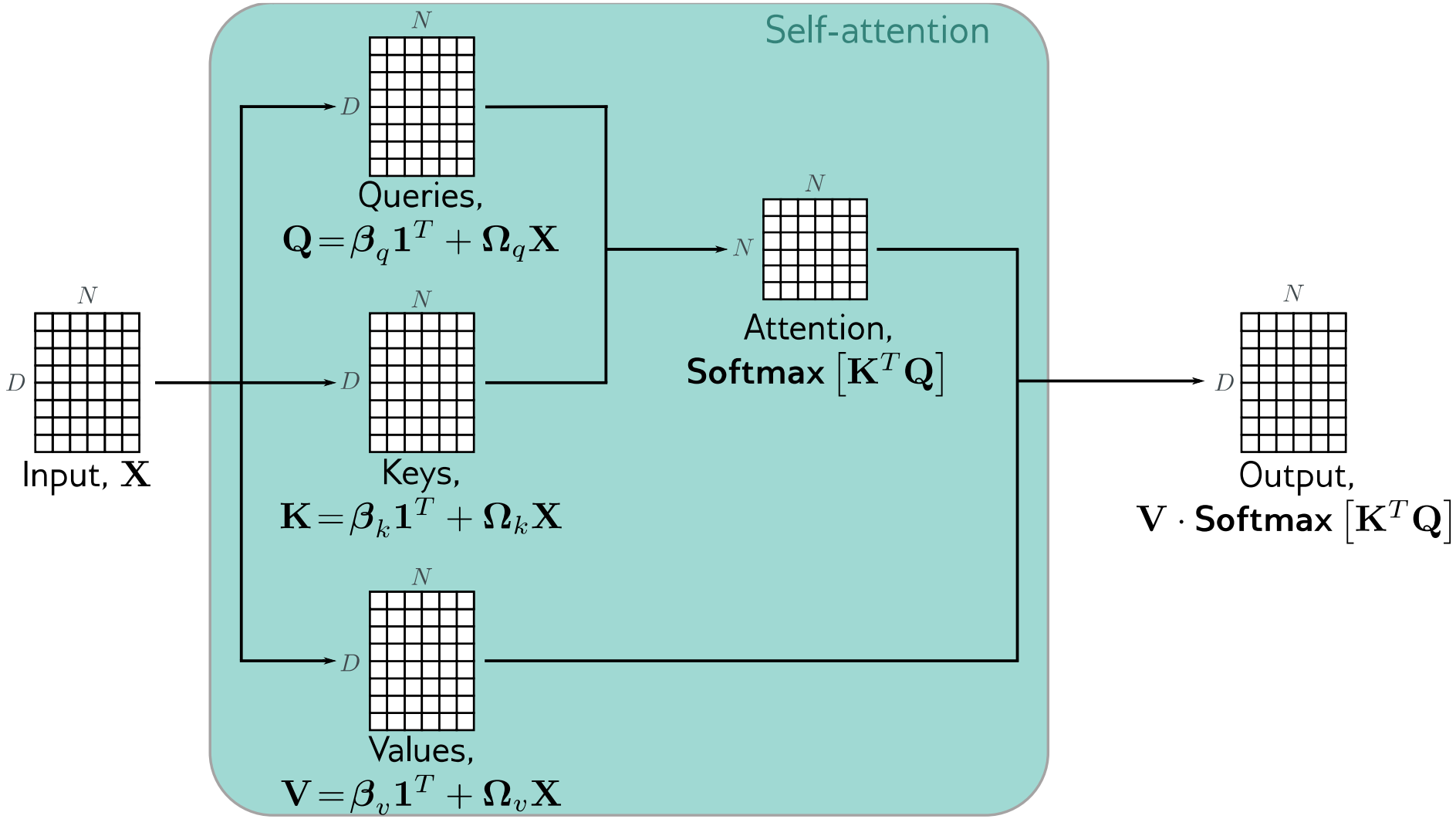
- Compute values, queries and keys:

$$\mathbf{V}[\mathbf{X}] = \boldsymbol{\beta}_v \mathbf{1}^\mathbf{T} + \boldsymbol{\Omega}_\mathbf{v}\mathbf{X}$$
$$\mathbf{Q}[\mathbf{X}] = \boldsymbol{\beta}_q \mathbf{1}^\mathbf{T} + \boldsymbol{\Omega}_\mathbf{q}\mathbf{X}$$
$$\mathbf{K}[\mathbf{X}] = \boldsymbol{\beta}_k \mathbf{1}^\mathbf{T} + \boldsymbol{\Omega}_\mathbf{k}\mathbf{X},$$

- Combine self-attentions

$$\mathbf{Sa}[\mathbf{X}] = \mathbf{V}[\mathbf{X}] \cdot \mathbf{Softmax}\Big[\mathbf{K}[\mathbf{X}]^T\mathbf{Q}[\mathbf{X}]\Big]$$

Self-attention in matrix form can be efficient



Is a single attention head enough?

Single softmax forces all 'considerations' to be weighted against each other.

Consider:

The bank near the river closed its doors early.

To understand sentence, must attend to:

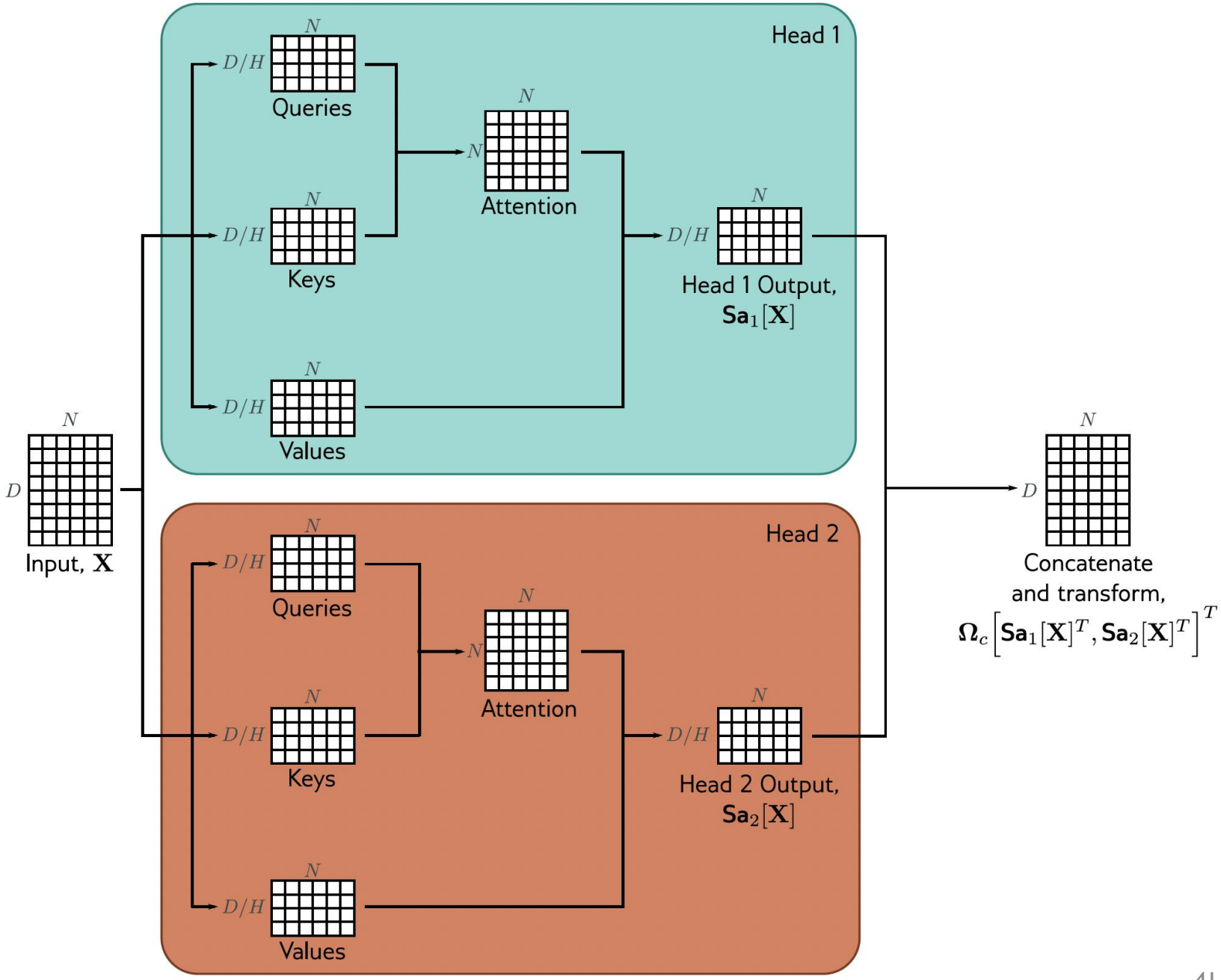
- meaning of *river* (not a financial bank),
- subject of verb *closed*,
- *its* – where *bank* is the antecedent.

Multi-head self attention

- Split input along dimension.
- Different $\boldsymbol{\Omega}, \boldsymbol{\beta}$ per head for each of the value, query, key transforms.
- Join with new transform $\boldsymbol{\Omega}_c$

Avoids single softmax;
not all 'considerations' need be weighted against each other.

More effective training.



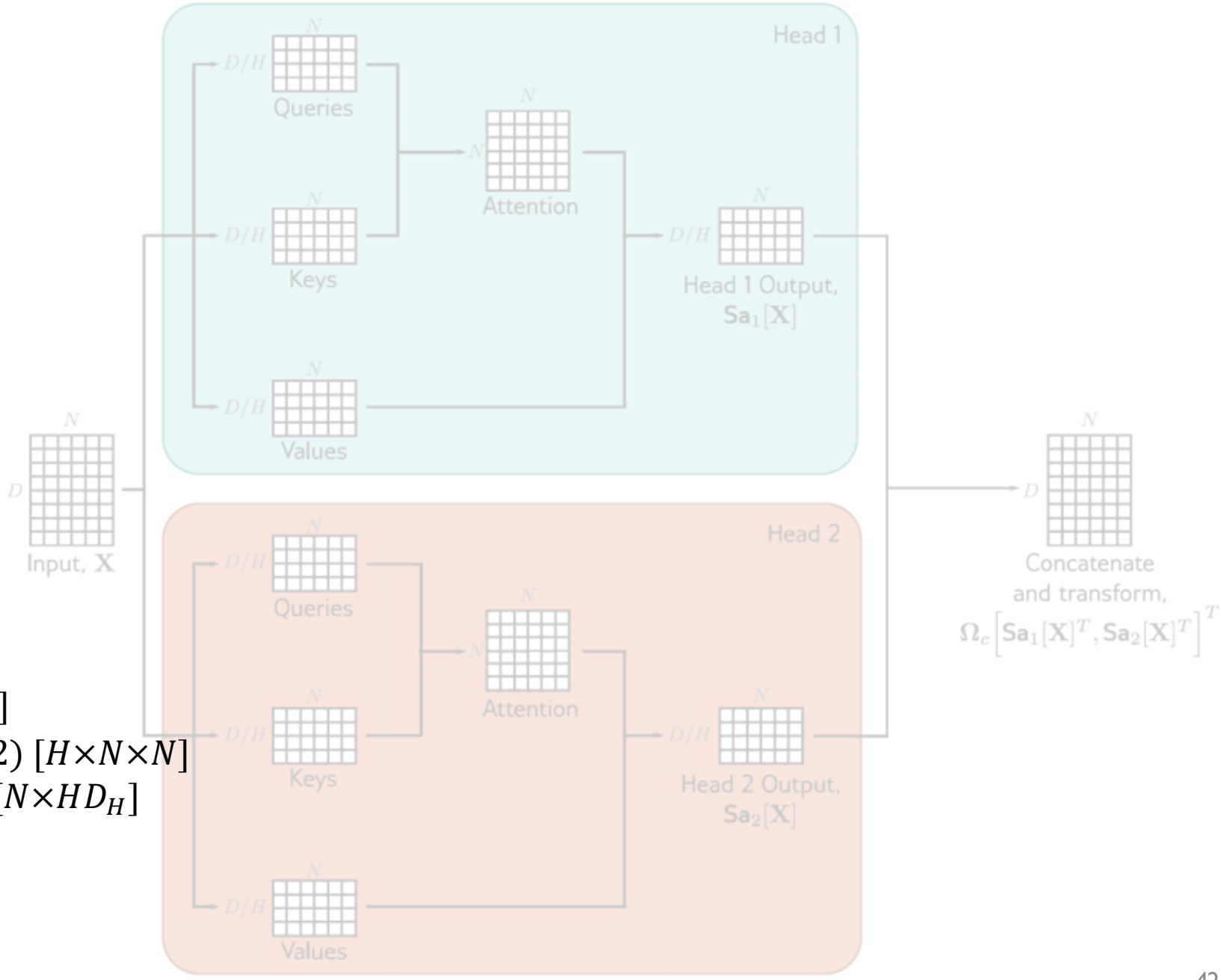
How many multiplies?

Four matrix multiples:

Input: $\mathbf{X}$ $[N \times D]$
Query matrix: $\mathbf{Q}$ $[D \times HD_H]$
Key matrix: $\mathbf{K}$ $[D \times HD_H]$
Value matrix: $\mathbf{V}$ $[D \times HD_H]$
Output matrix: $\mathbf{O}$ $[HD_H \times D]$

Queries: $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$ $[H \times N \times D_H]$
Keys: $\mathbf{K} = \mathbf{X}\mathbf{W}_K$ $[H \times N \times D_H]$
Values: $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{Q}\mathbf{K}^T / D_q$ $[H \times N \times N]$
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim} = 2)$ $[H \times N \times N]$
Head outputs: $\mathbf{Y} = \mathbf{A}\mathbf{V}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$
Outputs: $\mathbf{O} = \mathbf{Y}\mathbf{W}_O$ $[N \times D]$



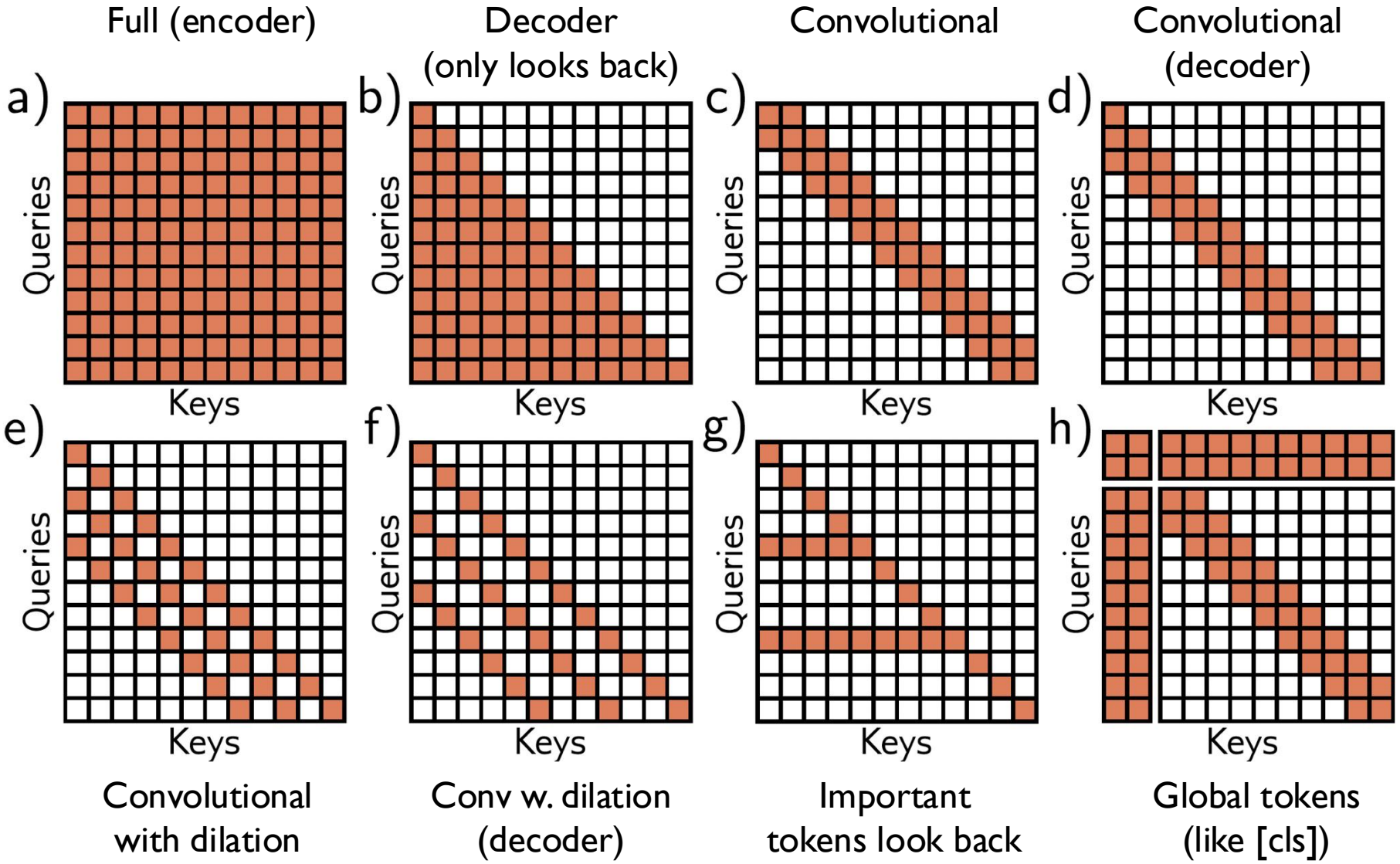
[Justin Johnson]

Cross-attention

Suppose I have multiple data modalities, e.g., images, text, as my input data vectors. Can I 'fuse' them?

Make each query produce one output which mixes information in the data vectors.

Self-attention interaction matrices



How many multiplies?

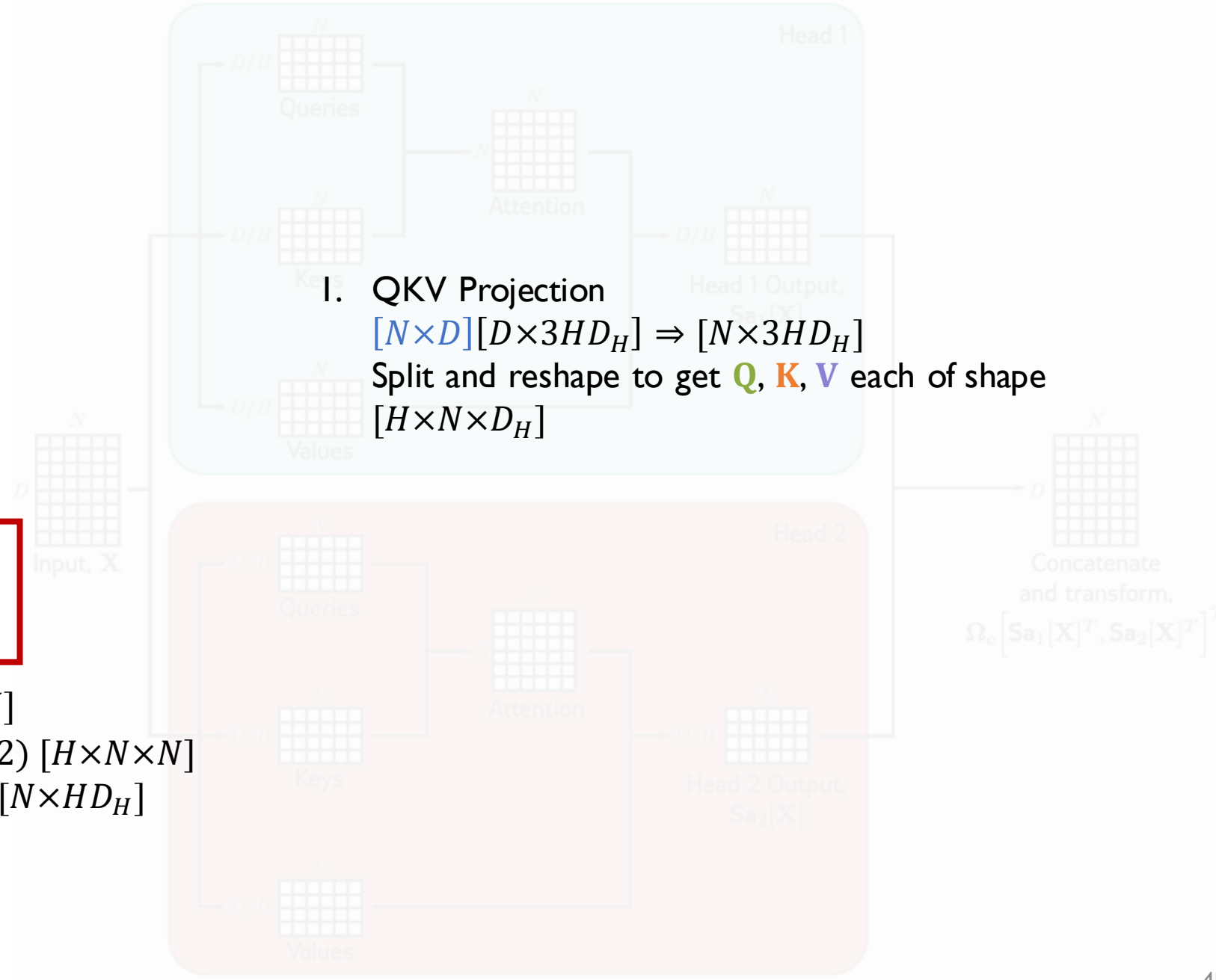
Four matrix multiples:

Input: $\mathbf{X}$ $[N \times D]$
Query matrix: $\mathbf{Q}$ $[D \times HD_H]$
Key matrix: $\mathbf{K}$ $[D \times HD_H]$
Value matrix: $\mathbf{V}$ $[D \times HD_H]$
Output matrix: $\mathbf{O}$ $[HD_H \times D]$

Queries: $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$ $[H \times N \times D_H]$
Keys: $\mathbf{K} = \mathbf{X}\mathbf{W}_K$ $[H \times N \times D_H]$
Values: $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{Q}\mathbf{K}^T / D_q$ $[H \times N \times N]$
Attention weights: $\mathbf{A} = \mathrm{softmax}(\mathbf{E}, \mathrm{dim} = 2)$ $[H \times N \times N]$
Head outputs: $\mathbf{Y} = \mathbf{A}\mathbf{V}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$
Outputs: $\mathbf{O} = \mathbf{Y}\mathbf{W}_O$ $[N \times D]$

1. QKV Projection
   $[N \times D][D \times 3HD_H] \Rightarrow [N \times 3HD_H]$
   Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$



How many multiplies?

Four matrix multiples:

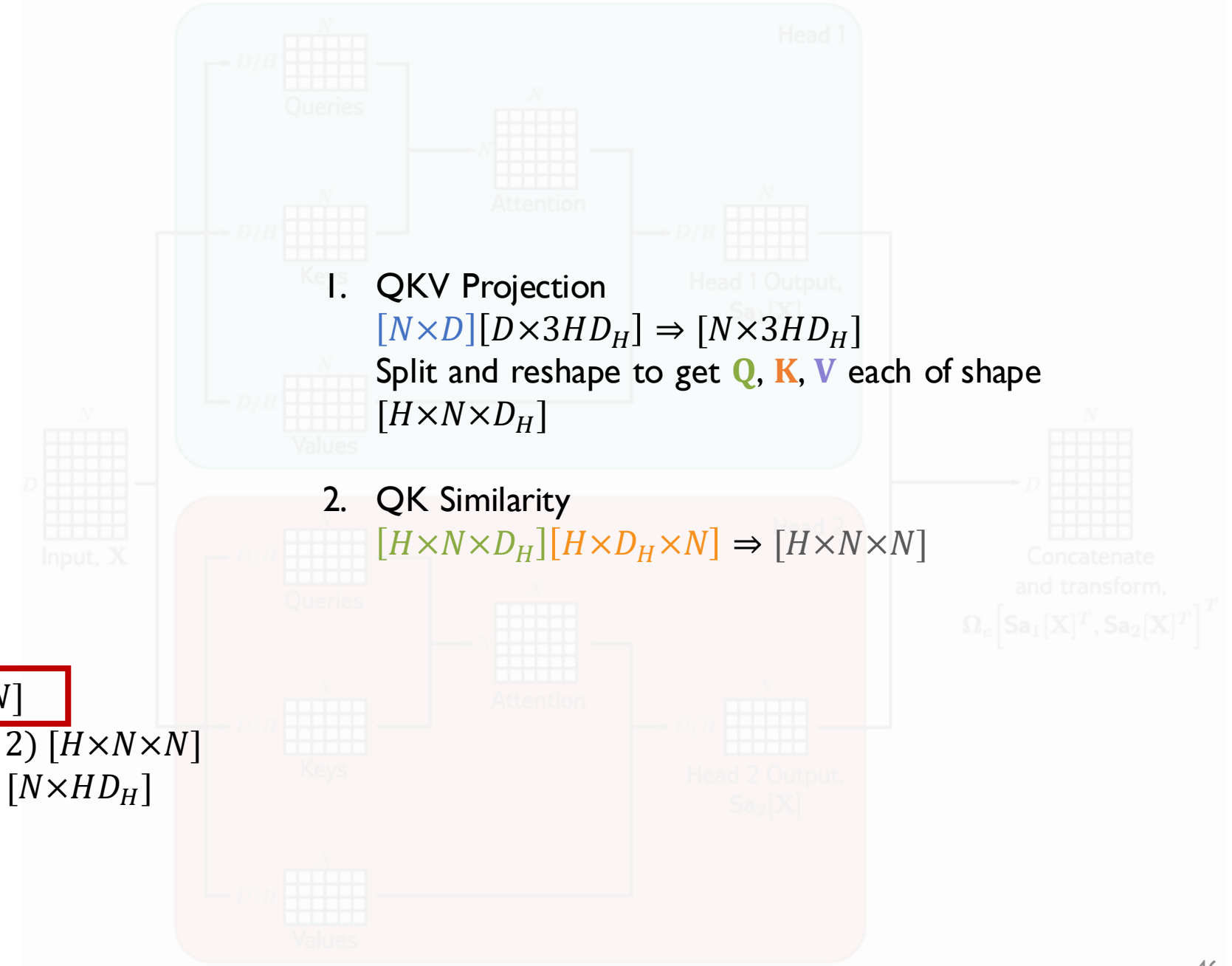
Input: $\mathbf{X}$ $[N \times D]$
Query matrix: $\mathbf{Q}$ $[D \times HD_H]$
Key matrix: $\mathbf{K}$ $[D \times HD_H]$
Value matrix: $\mathbf{V}$ $[D \times HD_H]$
Output matrix: $\mathbf{O}$ $[HD_H \times D]$

Queries: $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$ $[H \times N \times D_H]$
Keys: $\mathbf{K} = \mathbf{X}\mathbf{W}_K$ $[H \times N \times D_H]$
Values: $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{Q}\mathbf{K}^T / D_q$ $[H \times N \times N]$
Attention weights: $\mathbf{A} = \mathrm{softmax}(\mathbf{E}, \mathrm{dim} = 2)$ $[H \times N \times N]$
Head outputs: $\mathbf{Y} = \mathbf{A}\mathbf{V}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$
Outputs: $\mathbf{O} = \mathbf{Y}\mathbf{W}_O$ $[N \times D]$

1. QKV Projection
   $[N \times D][D \times 3HD_H] \Rightarrow [N \times 3HD_H]$
   Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity
   $[H \times N \times D_H][H \times D_H \times N] \Rightarrow [H \times N \times N]$



[Justin Johnson]

How many multiplies?

Four matrix multiples:

Input:	$\mathbf{X}$ $[N \times D]$
Query matrix:	$\mathbf{Q}$ $[D \times HD_H]$
Key matrix:	$\mathbf{K}$ $[D \times HD_H]$
Value matrix:	$\mathbf{V}$ $[D \times HD_H]$
Output matrix:	$\mathbf{O}$ $[HD_H \times D]$

Queries:	$\mathbf{Q} = \mathbf{X}\mathbf{W}_\mathrm{Q}$ $[H \times N \times D_H]$
Keys:	$\mathbf{K} = \mathbf{X}\mathbf{W}_\mathrm{K}$ $[H \times N \times D_H]$
Values:	$\mathbf{V} = \mathbf{X}\mathbf{W}_\mathrm{V}$ $[H \times N \times D_H]$
Similarities:	$\mathbf{E} = \mathbf{Q}\mathbf{K}^\mathrm{T} / D_q$ $[H \times N \times N]$
Attention weights:	$\mathbf{A} = \mathrm{softmax}(\mathbf{E}, \mathrm{dim} = 2)$ $[H \times N \times N]$
Head outputs:	$\mathbf{Y} = \mathbf{A}\mathbf{V}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$
Outputs:	$\mathbf{O} = \mathbf{Y}\mathbf{W}_\mathrm{O}$ $[N \times D]$

1. QKV Projection
   $[N \times D][D \times 3HD_H] \Rightarrow [N \times 3HD_H]$
   Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity
   $[H \times N \times D_H][H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V Weighting
   $[H \times N \times N][H \times N \times D_H] \Rightarrow [H \times N \times D_H]$
   Reshape to $[N \times HD_H]$

How many multiplies?

Four matrix multiples:

Input: $\mathbf{X}$ $[N \times D]$
Query matrix: $\mathbf{Q}$ $[D \times HD_H]$
Key matrix: $\mathbf{K}$ $[D \times HD_H]$
Value matrix: $\mathbf{V}$ $[D \times HD_H]$
Output matrix: $\mathbf{O}$ $[HD_H \times D]$

Queries: $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$ $[H \times N \times D_H]$
Keys: $\mathbf{K} = \mathbf{X}\mathbf{W}_K$ $[H \times N \times D_H]$
Values: $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{Q}\mathbf{K}^T / D_q$ $[H \times N \times N]$
Attention weights: $\mathbf{A} = \mathrm{softmax}(\mathbf{E}, \mathrm{dim} = 2)$ $[H \times N \times N]$
Head outputs: $\mathbf{Y} = \mathbf{A}\mathbf{V}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$
Outputs: $\mathbf{O} = \mathbf{Y}\mathbf{W}_O$ $[N \times D]$

1. QKV Projection
   $[N \times D][D \times 3HD_H] \Rightarrow [N \times 3HD_H]$
   Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity
   $[H \times N \times D_H][H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V Weighting
   $[H \times N \times N][H \times N \times D_H] \Rightarrow [H \times N \times D_H]$
   Reshape to $[N \times HD_H]$

4. Output Projection
   $[N \times HD_H][HD_H \times D] \Rightarrow [N \times D]$

How many multiplies?

Four matrix multiples:

Input: $\mathbf{X}$ $[N \times D]$
Query matrix: $\mathbf{Q}$ $[D \times HD_H]$
Key matrix: $\mathbf{K}$ $[D \times HD_H]$
Value matrix: $\mathbf{V}$ $[D \times HD_H]$
Output matrix: $\mathbf{O}$ $[HD_H \times D]$

Queries: $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$ $[H \times N \times D_H]$
Keys: $\mathbf{K} = \mathbf{X}\mathbf{W}_K$ $[H \times N \times D_H]$
Values: $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{Q}\mathbf{K}^T / D_q$ $[H \times N \times N]$
Attention weights: $\mathbf{A} = \mathrm{softmax}(\mathbf{E}, \mathrm{dim} = 2)$ $[H \times N \times N]$
Head outputs: $\mathbf{Y} = \mathbf{A}\mathbf{V}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$
Outputs: $\mathbf{O} = \mathbf{Y}\mathbf{W}_O$ $[N \times D]$

D is large and fixed (e.g., 384); H is small and fixed (e.g., 8, 12).
Compute + memory is dominated by $O(N^2)$ term.

Flash Attention: Compute 2+3 dynamically to never store $\mathbf{E}/\mathbf{A}$.

1. QKV Projection
   $[N \times D][D \times 3HD_H] \Rightarrow [N \times 3HD_H]$
   Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity
   $[H \times N \times D_H][H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V Weighting
   $[H \times N \times N][H \times N \times D_H] \Rightarrow [H \times N \times D_H]$
   Reshape to $[N \times HD_H]$

4. Output Projection
   $[N \times HD_H][HD_H \times D] \Rightarrow [N \times D]$

Position encoding Π

Self-attention is equivariant to permuting sequence order.

But order is sometimes important in sequences:

The man ate the fish

vs.

The fish ate the man

Position encoding Π

Design 1D function that is unique per sequence position.

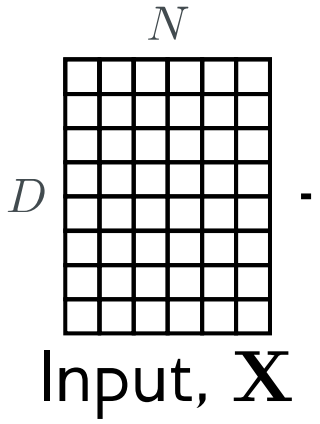
For pos position in sequence:

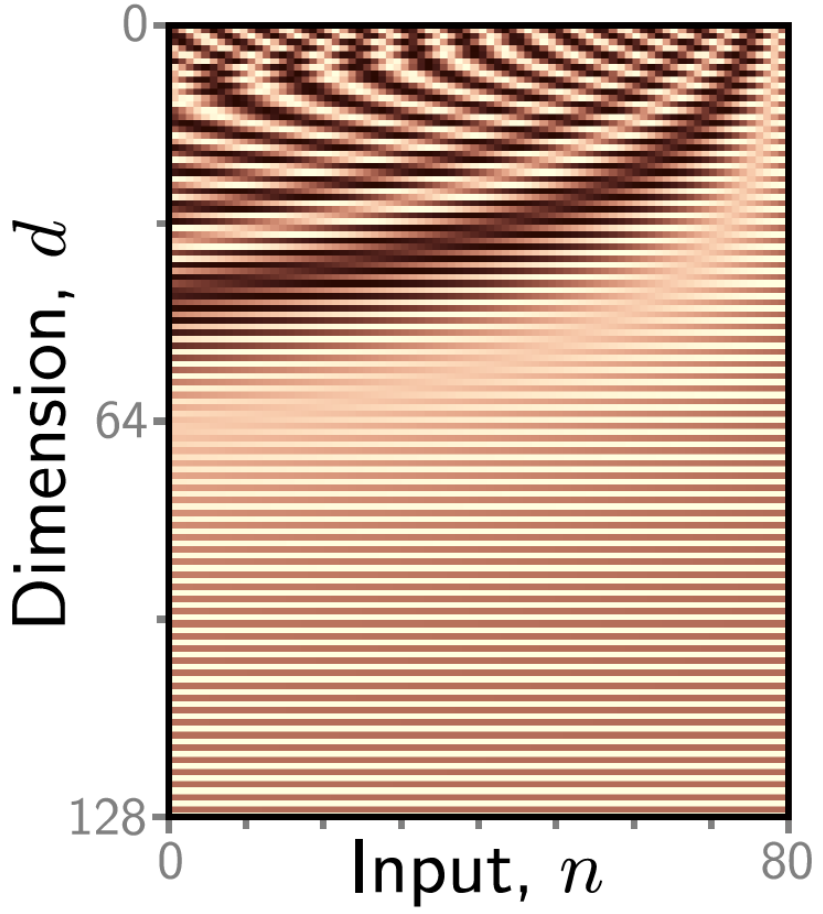
$$PE(pos, d) = \sin\left(\frac{pos}{10{,}000^{\frac{d}{D}}}\right)$$

$$PE(pos, d+1) = \cos\left(\frac{pos}{10{,}000^{\frac{d}{D}}}\right)$$

Is a Fourier encoding!
$d = 2i =$ which frequency band dimension pair encodes.

Featurizes position! Makes learning on positions easier because they have extent across the vector D.





Position encoding $\mathbf{\Pi}$

Design function that is unique per sequence position.

Then *add* (or concat) to values, keys, queries:

$$\mathbf{Sa}[\mathbf{X}] = \mathbf{V} \cdot \mathbf{Softmax}[\mathbf{K}^T\mathbf{Q}]$$



$$\mathbf{Sa}[\mathbf{X}] = (\mathbf{V} + \mathbf{\Pi}) \cdot \mathbf{Softmax}[(\mathbf{K} + \mathbf{\Pi})^T(\mathbf{Q} + \mathbf{\Pi})]$$

Position encoding Π tricks I

- Learn the PE encoding.

- May learn to emphasize layout info in data, e.g., more encoding precision needed at edges of images.
- Unlike fixed PE, depends upon image resolution;
  must interpolate to new size

Position encoding Π tricks 2

Design 2D function that is unique per sequence position.

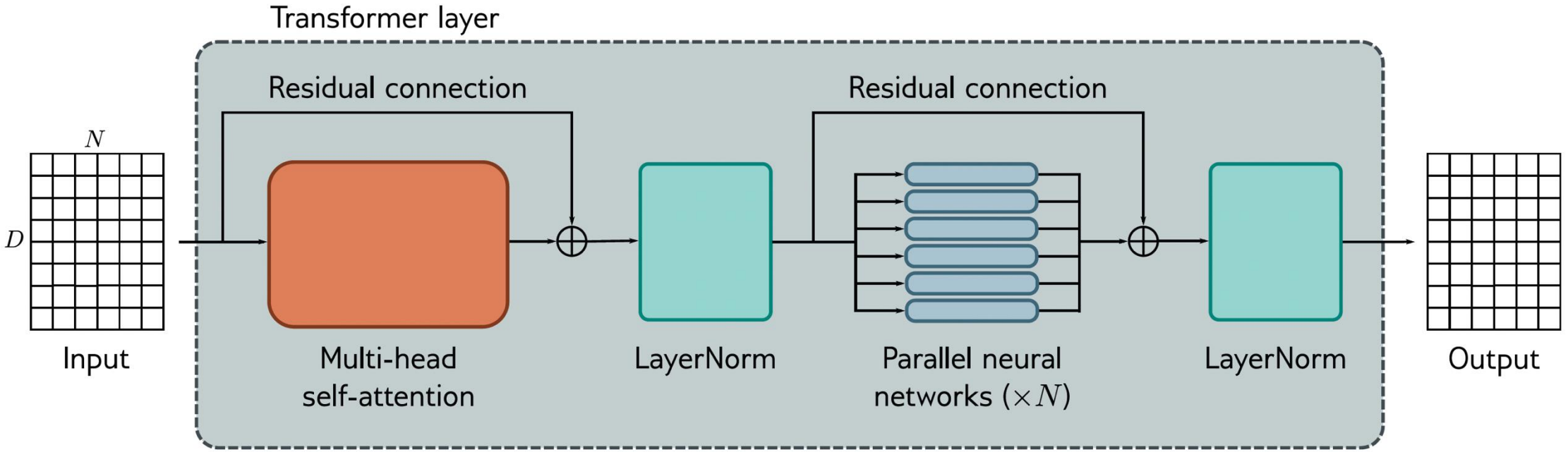
As before, but split D in two:

- [0,D/2) encodes row in image
- [D/2,D) encodes column in image.

Model becomes *independent* of input sequence order!

[ViT Masked Auto Encoder]

The transformer layer



[Vaswani et al., Attention is All You Need 2017]

ImageGPT (OpenAI)

Each pixel is a token.

Image built auto-regressively as next-pixel prediction in Western order.

9-bit indexed color (512 values only)

48x48 images

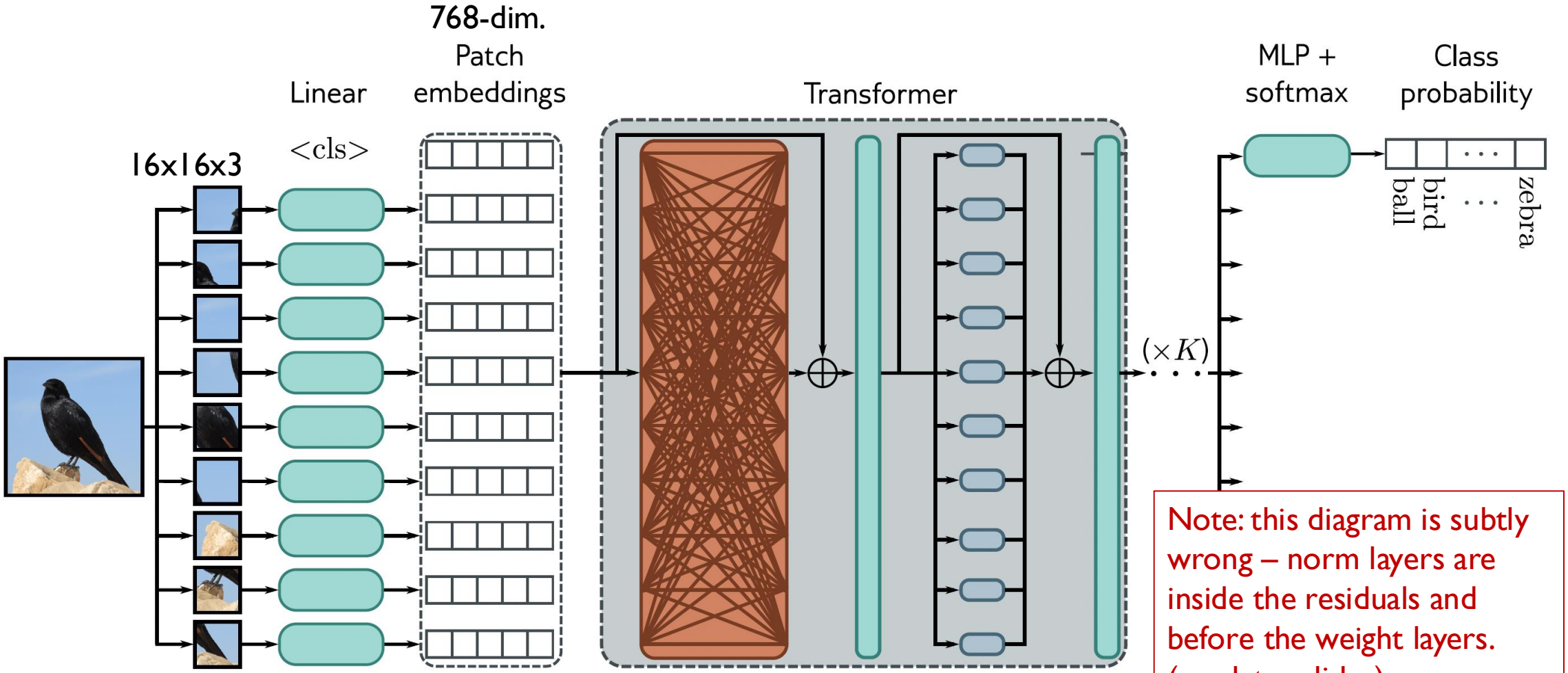


Conditional generation

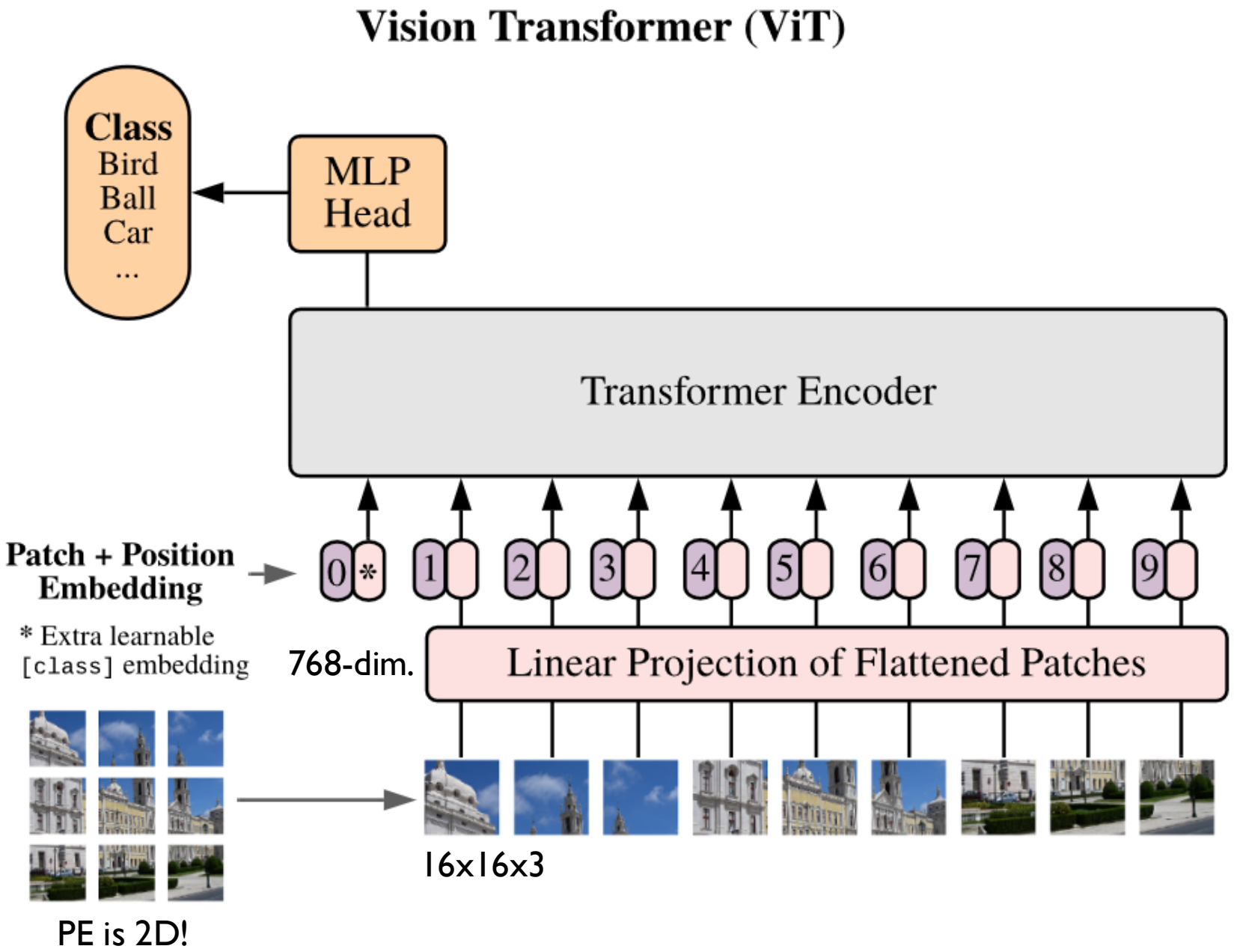
Top-left pixel: Class-likely sample.

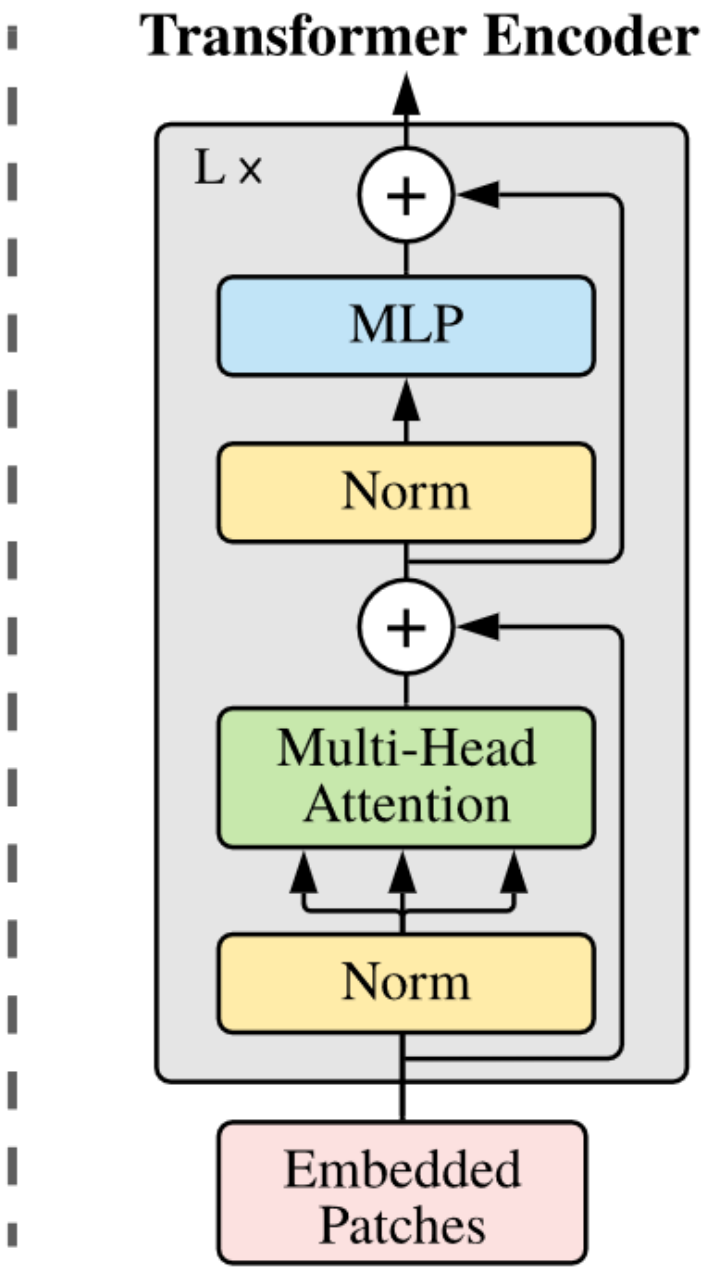
Variations on completion

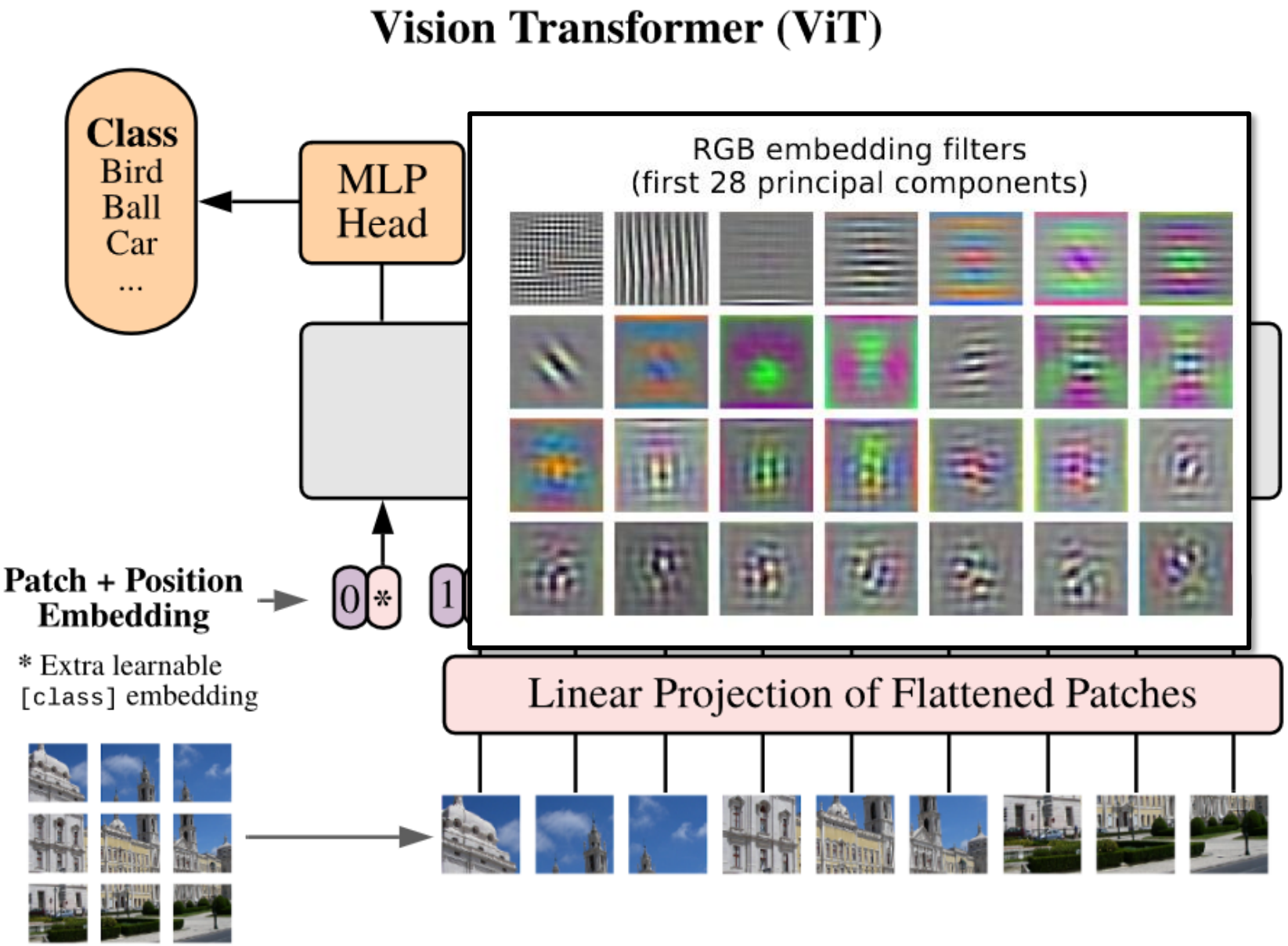
Vision Transformer [Dosovitskiy et al. ICLR 2021]

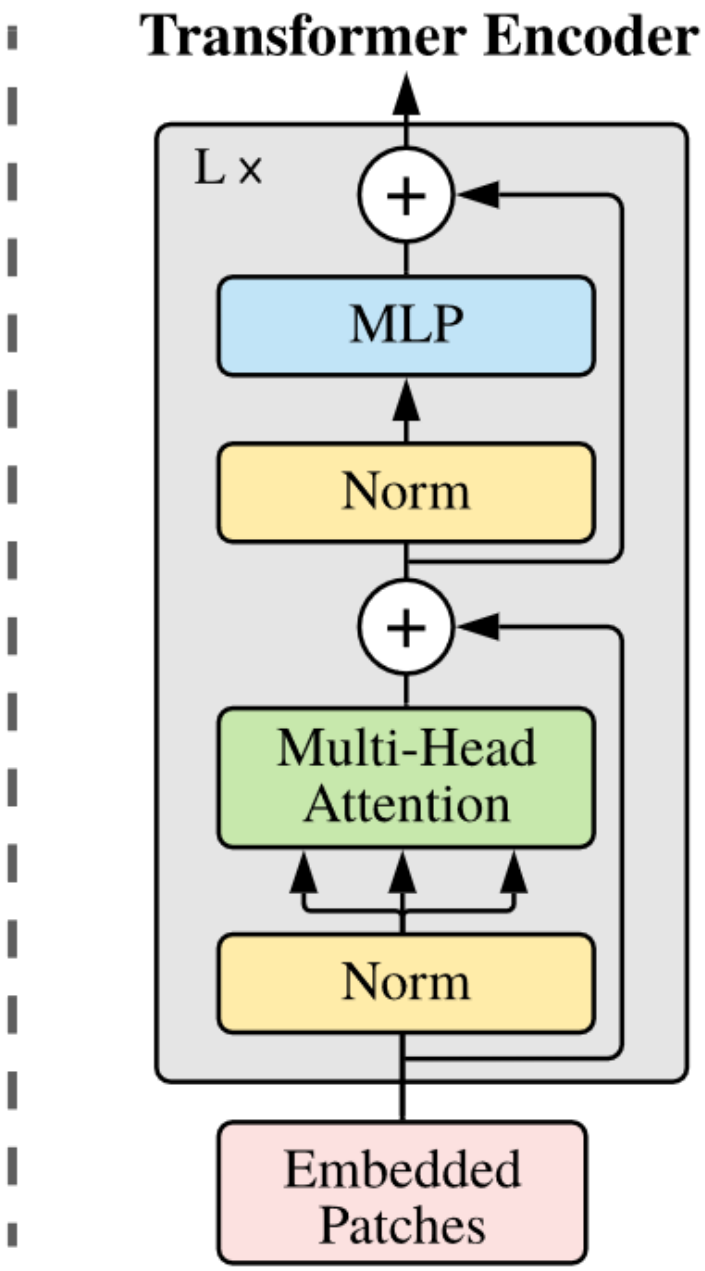


Trained on supervised task with 18,000 classes on 303 M images (Google's private JFT dataset).


Vision Transformer (ViT)
Class
Bird
Ball
Car
...
MLP Head
Transformer Encoder
Patch + Position Embedding
* Extra learnable [class] embedding
0 * 1 2 3 4 5 6 7 8 9
768-dim.
Linear Projection of Flattened Patches
16x16x3
PE is 2D!


Transformer Encoder
L x
MLP
Norm
Multi-Head Attention
Norm
Embedded Patches


Vision Transformer (ViT)
Class
Bird
Ball
Car
...
MLP
Head
RGB embedding filters
(first 28 principal components)
Patch + Position
Embedding
0
*
1
* Extra learnable
[class] embedding
Linear Projection of Flattened Patches


Transformer Encoder
L x
MLP
Norm
Multi-Head
Attention
Norm
Embedded
Patches

So what is [cls]?

A free *learnable* vector prepended to the patch sequence (position 0).

- No image content.
- Attends to all patches through self-attention layers.
- Classification head attached only to last layer [cls] as "image embedding".
- [cls] feature vector is optimized by cross-entropy loss.

Different from Global Average Pooling in CNNs:

- Both produce a single feature vector that can be used for classification.
- [cls] is feature learned through attention weights on other patch features.
- GAP uses fixed mean on layer patch features. Some ViTs also do GAP.

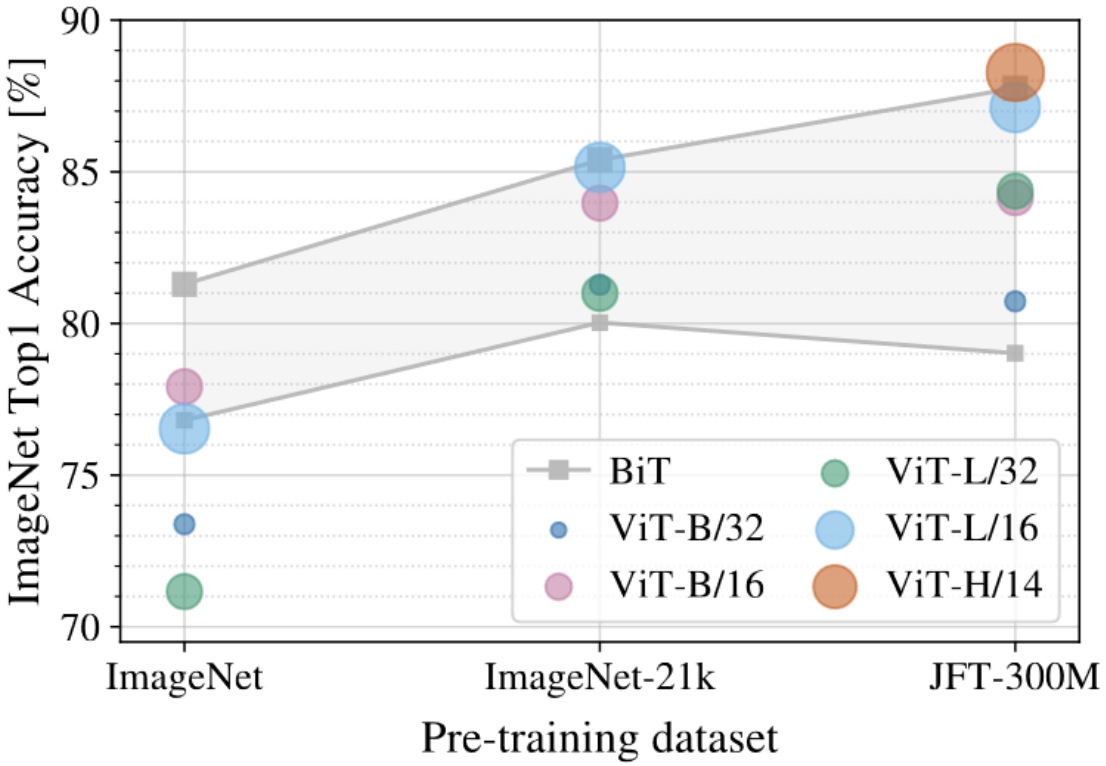
Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Table 1: Details of Vision Transformer model variants.

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Table 1: Details of Vision Transformer model variants.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21K (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	**90.72** ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	**99.50** ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	–
CIFAR-100	**94.55** ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	–
Oxford-IIIT Pets	**97.56** ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	–
Oxford Flowers-102	99.68 ± 0.02	**99.74** ± 0.00	99.61 ± 0.02	99.63 ± 0.03	–
VTAB (19 tasks)	**77.63** ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k



"When trained on mid-sized datasets such as ImageNet, such models yield modest accuracies of a few percentage points below ResNets of comparable size. This seemingly discouraging outcome may be expected: Transformers lack some of the inductive biases inherent to CNNs, such as translation equivariance and locality, and therefore do not generalize well when trained on insufficient amounts of data.

However, the picture changes if the models are trained on larger datasets (14M-300M images). We find that large scale training trumps inductive bias."

Dosovitskiy et al. 2021

ConvNeXT (2022) again showed equivalent perf.

Does PE help?

Yes, a bit.

Pos. Emb.	Default/Stem	Every Layer	Every Layer-Shared
No Pos. Emb.	0.61382	N/A	N/A
1-D Pos. Emb.	0.64206	0.63964	0.64292
2-D Pos. Emb.	0.64001	0.64046	0.64022
Rel. Pos. Emb.	0.64032	N/A	N/A

Table 8: Results of the ablation study on positional embeddings with ViT-B/16 model evaluated on ImageNet 5-shot linear.

Attention visualized

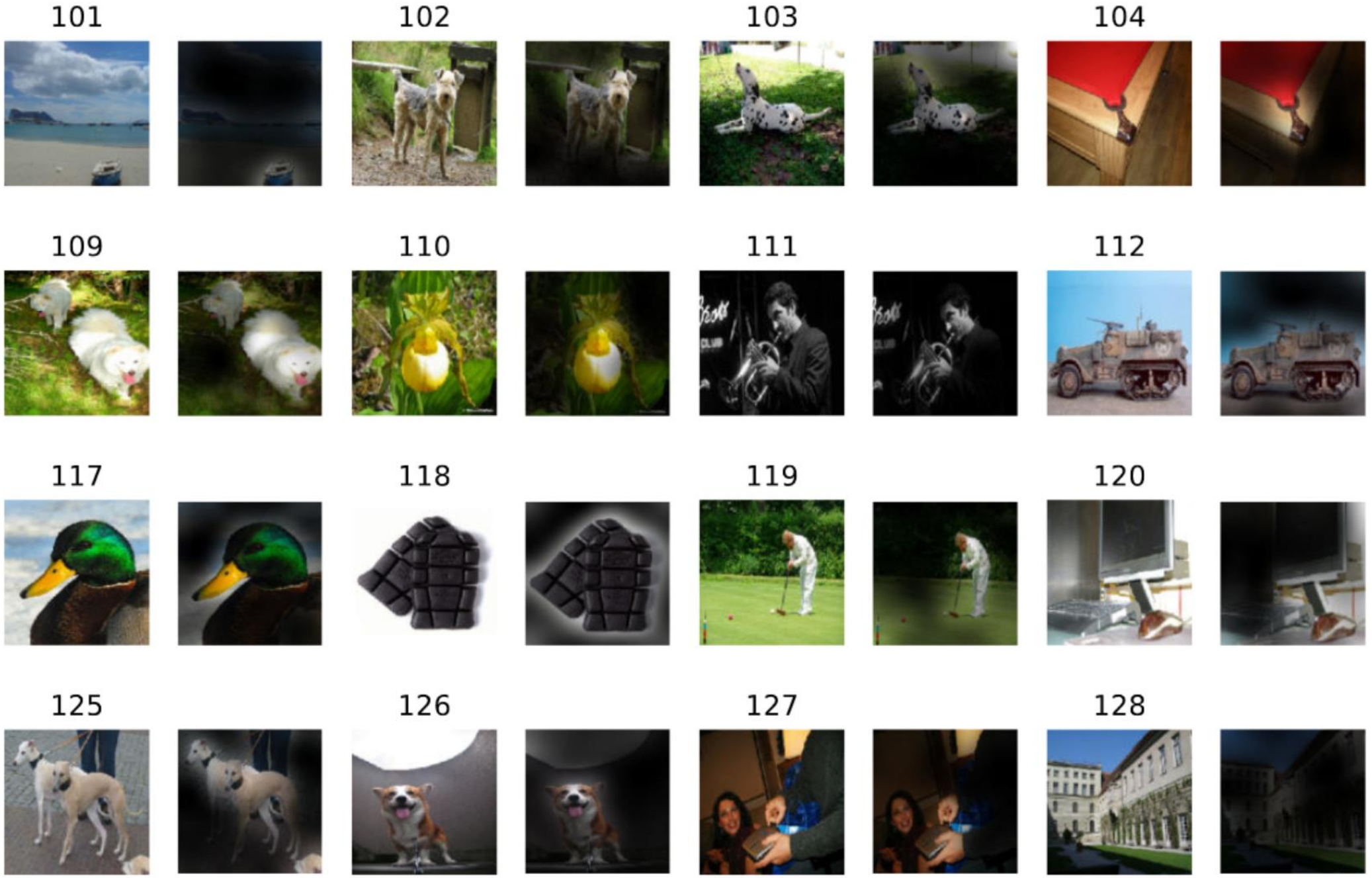
[cls] token attends to all tokens (patches) in sequence.

Take [cls] from some layer (e.g., last one).

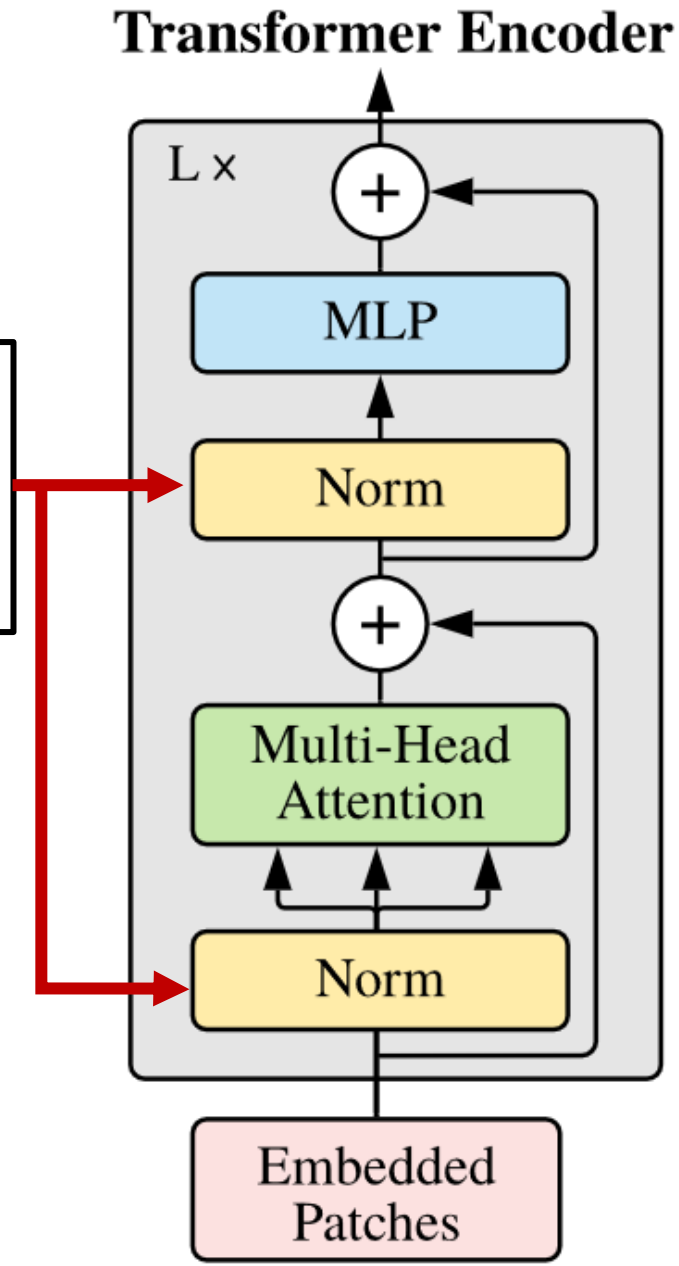
Consider [cls] attention weights to each patch.

Reshape back into low-res 2D image.

Upscale + fade to black based on attention value.

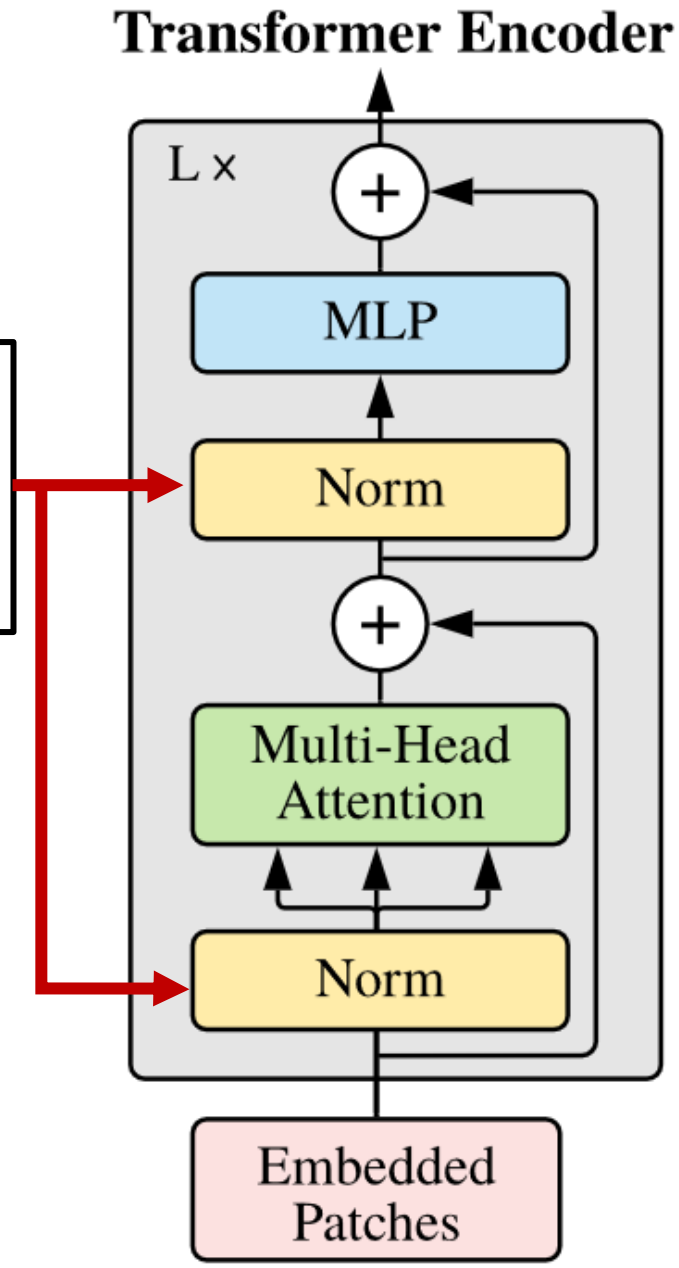


Layer norm now before MLP and attention:
Allows residual to learn identity.



[Baevski & Auli, "Adaptive Input Representations for Neural Language Modeling", arXiv 2018]

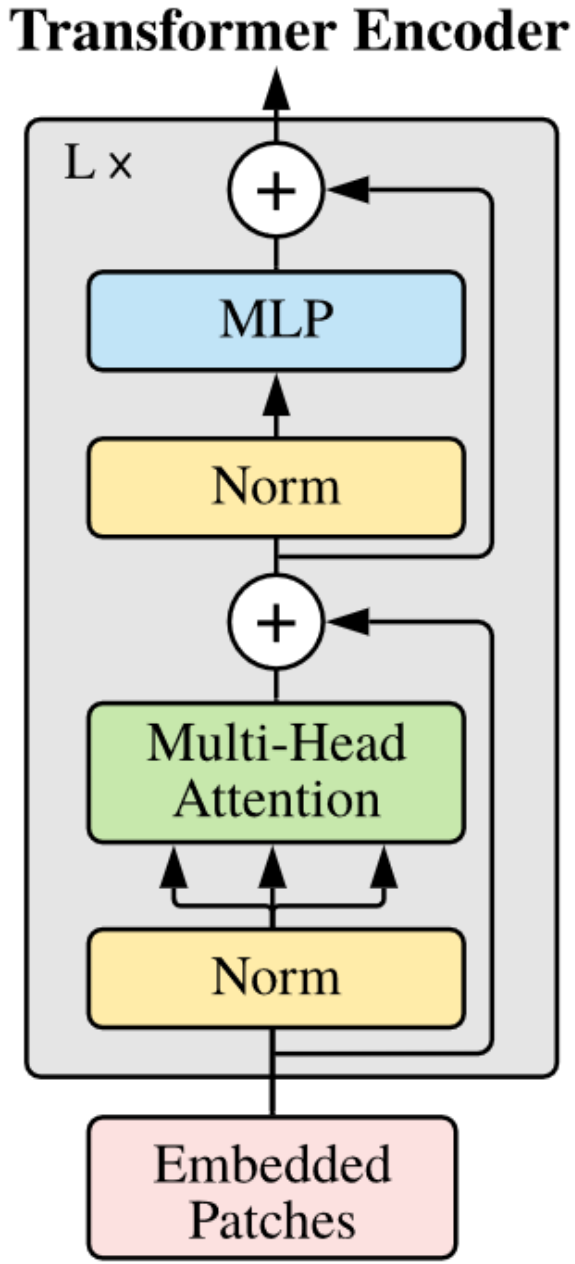
Layer norm now before MLP and attention:
Allows residual to learn identity.



[Baevski & Auli, "Adaptive Input Representations for Neural Language Modeling", arXiv 2018]

Mixture of Experts (MoE):

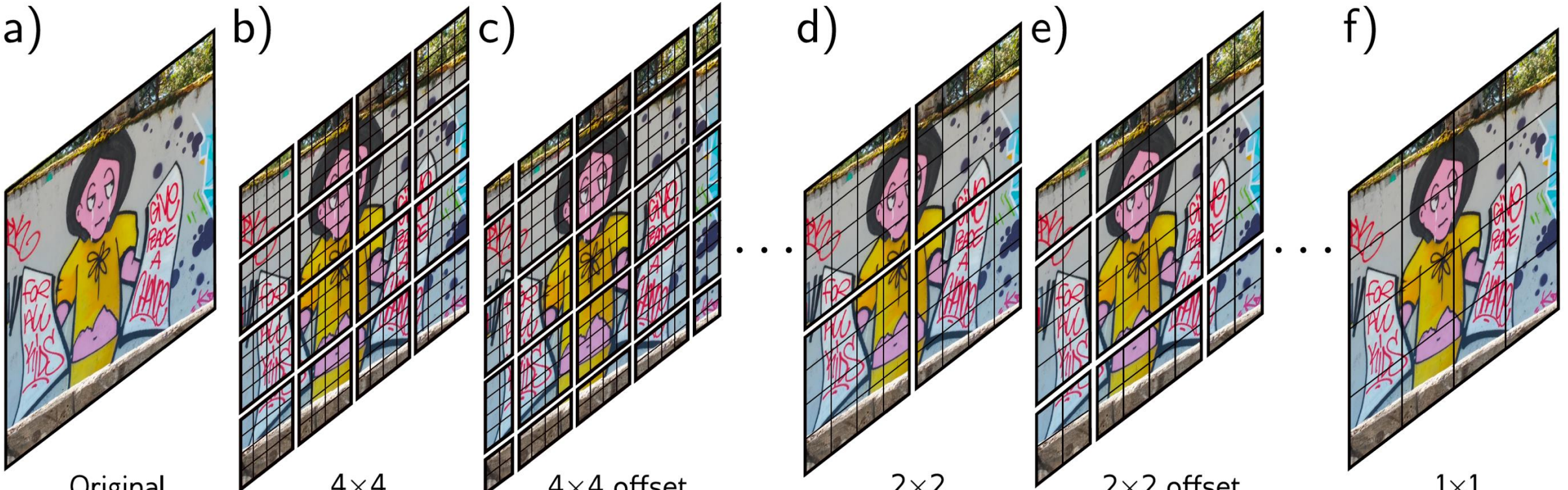
- Learn E separate sets of MLP weights ('experts') in each block
- Each token gets routed to A < E *active experts*.
- Increases params by E but cost by A.
- Very common in state-of-the-art big models.



[MoE - Shazeer et al, "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer", 2017]

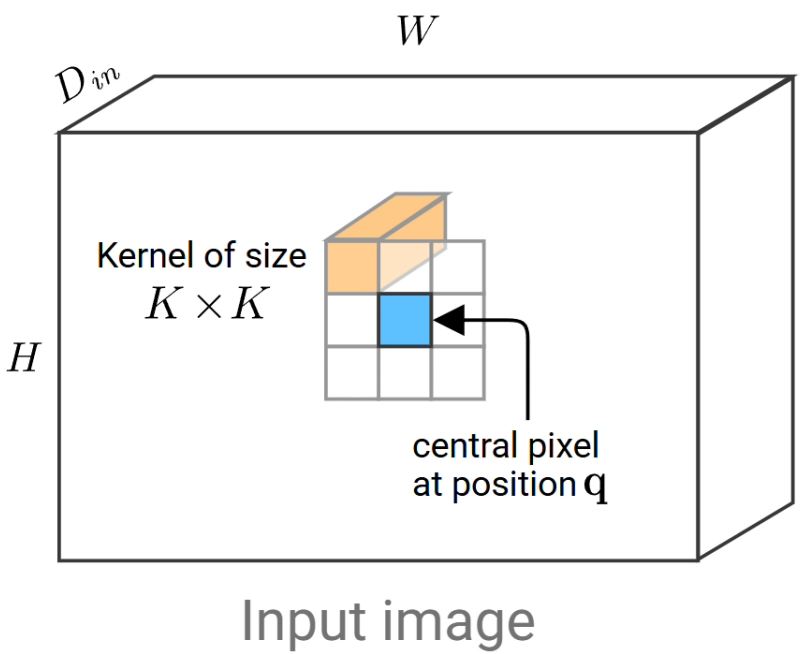
Shifted window (Swin) Transformer

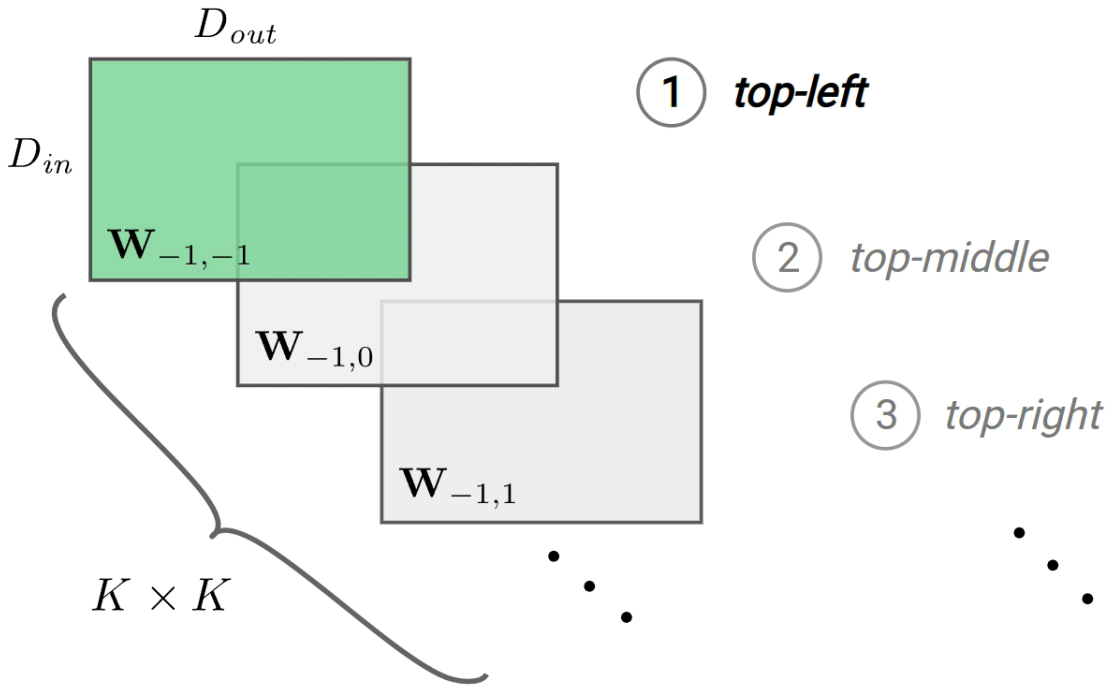
How to add the 'hierarchy' (scale space pyramid) back?



[Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, Baining Guo; Hierarchical Vision Transformer, CVPR 2021]

Self-attention can express convolution





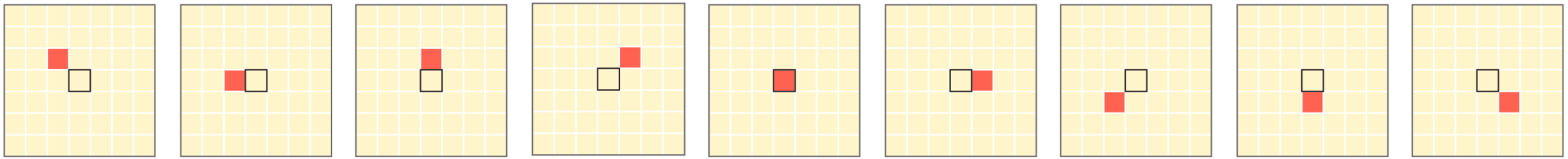
Apply linear map on each pixel individually and sum

$$\mathbf{Y}_{q,:} = \sum_{\Delta \in \mathbb{\Delta}} \mathbf{X}_{q+\Delta,:} \mathbf{W}_{\Delta,:,:} + \boldsymbol{b}$$

[Cordonnier et al., On the Relationship between Self-Attention and Convolutional Layers; ICLR 2020]

Self-attention can express convolution

Intuitive idea:

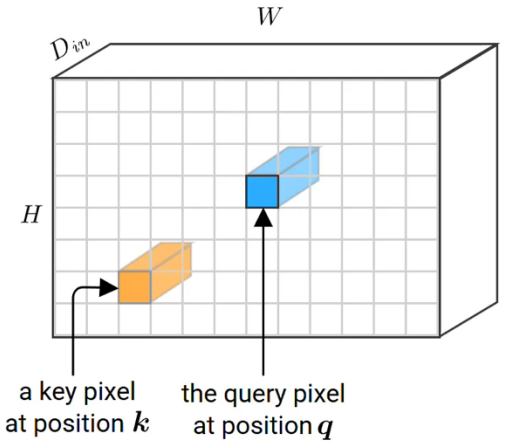


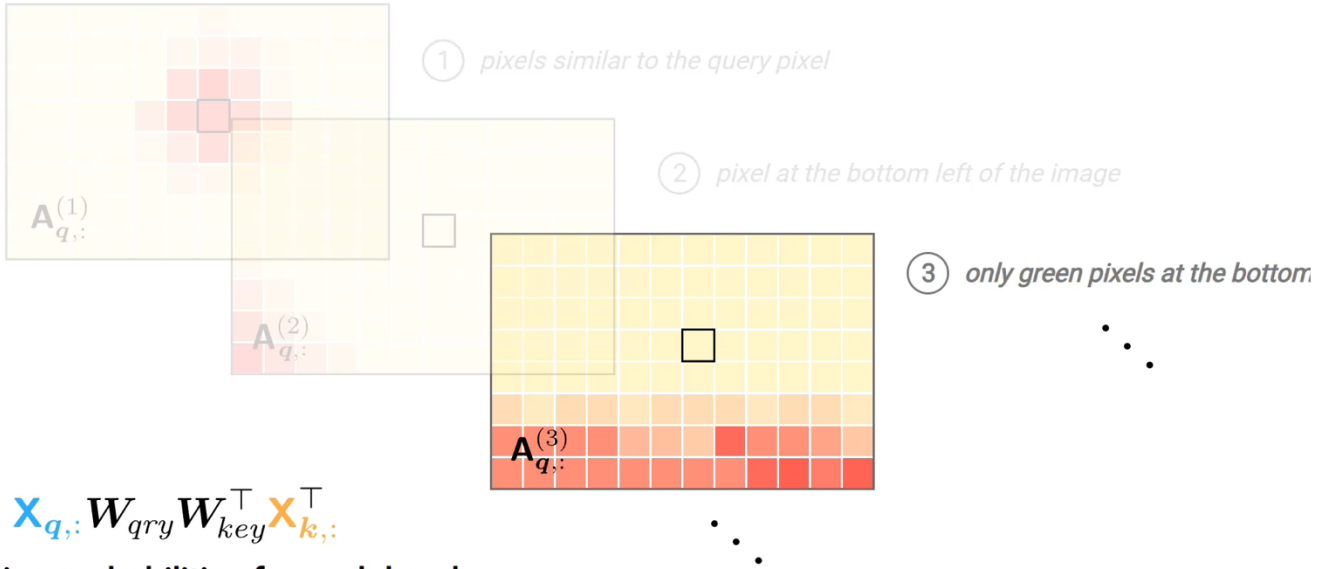
Black outline pixel is the query.

Suppose each head attended to just the red pixel,
then we can compose convolution.

- # attention heads ≥ # elements in convolution kernel
- What about translation equivariance? Use *relative* positional encoding!
  Inject relative distance dynamically into self-attention.

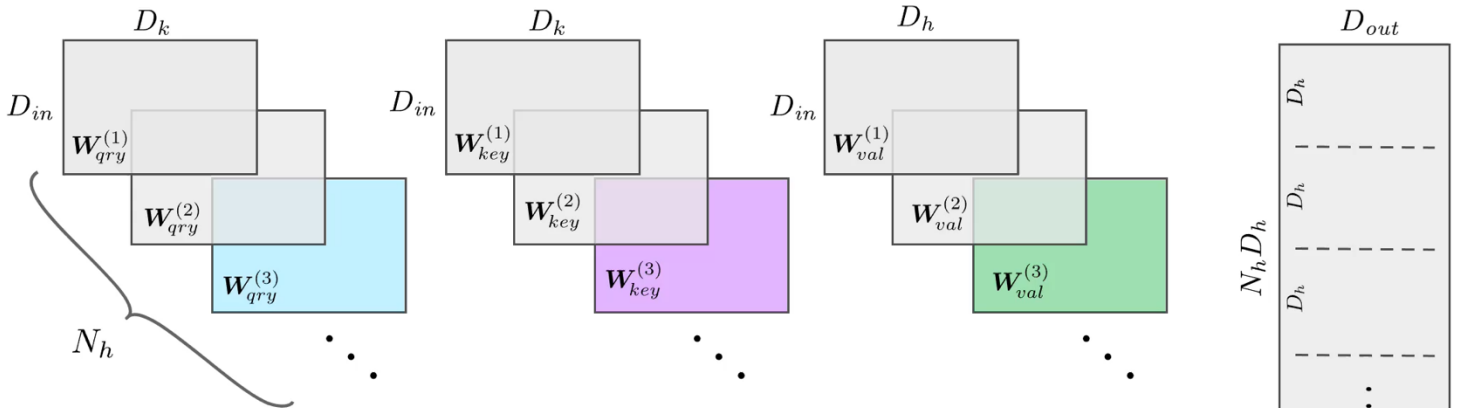
[Cordonnier, Loukas, Jaggi, ICLR 2020; Relative positional encoding: Dai et al. TransformerXL, ACL 2019]





$$\mathbf{A}_{q,:} = \mathbf{X}_{q,:} \boldsymbol{W}_{qry} \boldsymbol{W}_{key}^{\top} \mathbf{X}_{k,:}^{\top}$$

Compute attention probabilities for each head



Compute heads value $\mathbf{H}^{(h)}_{q,:} = \sum_{k' \in [W] \times [H]} \text{softmax}(\mathbf{A}^{(h)}_{q,:})_{k'} \; \mathbf{X}_{k',:} \boldsymbol{W}^{(h)}_{val}$

Compute output value $\mathbf{Y}_{q,:} = \text{concat}(\mathbf{H}^{(1)}_{q,:}, \ldots, \mathbf{H}^{(N_h)}_{q,:}) \boldsymbol{W}_{out} + \boldsymbol{b}_{out}$

Self-attention can express convolution–does it?

Fully-attentional model
6 layers with 9 heads each.

Trained on CIFAR-10 classification;
Reaches 94% accuracy



[Cordonnier, Loukas, Jaggi, ICLR 2020]

Demo

https://epfml.github.io/attention-cnn/